

COSC 499 Capstone Project Final Report

Intelligent Exam Variant
and Analytics System

Dhruv Bhardwaj, Tanner Bjorgan, Qingyun Qian
Peter Szabo, Ziyi Xia, Jasper Xiang, Luke Yin

August 8, 2025

Contents

1	Overview	1
1.1	Motivation	1
1.2	Scope and Boundaries	1
1.3	Unique Value Proposition	1
1.4	User Groups and Their Needs	2
2	System Architecture	3
2.1	Overall Architecture	3
2.1.1	Key Technical Achievements	3
2.1.2	Component Responsibilities	4
2.1.3	Advanced Security and Data Integrity	5
2.2	Technology Stack	5
2.3	Data Flow Diagrams	6
3	System Features	8
3.1	Overview of Features	8
3.2	User Requirements Implementation	8
3.2.1	UR1.1 - Secure login	8
3.2.2	UR1.2 - Instructor account management	8
3.2.3	UR1.3 - Global settings configuration	9
3.2.4	UR1.4 - System usage and audit logs	9
3.2.5	UR1.5 - Privacy and data retention	9
3.2.6	UR2.1 - Secure login with role-based access and password recovery . .	10
3.2.7	UR2.2 - Build question bank	10
3.2.8	UR2.3 - Add questions with correct answers to question bank	11
3.2.9	UR2.4 - Import questions from external files with metadata	11
3.2.10	UR2.5 - Create/manage course	12
3.2.11	UR2.6 - Add questions/question bank to exam	12
3.2.12	UR2.7 - Add/edit/manage exam variants from course	12
3.2.13	UR2.8 - Add/modify/manage student rosters for course	13
3.2.14	UR2.9 - Configure exam parameters	13
3.2.15	UR2.10(a) - Generate exams with single randomized variant	14
3.2.16	UR2.10(b) - Generate exams with multiple randomized variants . . .	14
3.2.17	UR2.11 - Export exam docx/PDFs (print-ready)	15
3.2.18	UR2.12 - Analyze exam statistics for answer similarity per question .	15
3.2.19	UR2.13 - Track student-variant assignment for answer matching . . .	15
3.2.20	UR2.14 - Define, edit, and validate answer keys per variant	16
3.2.21	UR2.15 - Import results from OMR systems	16
3.2.22	UR2.16 - Track student version assignment for scoring alignment . . .	17
3.2.23	UR2.17 - Generate student reports with item-level analysis	17
3.2.24	UR2.18 - Compare performance across exams and years	18
3.2.25	UR2.19 - Export results (CSV, PDF)	18
3.2.26	UR2.20 - Collaborate with co-instructors/TAs within course	18

3.2.27	UR2.21 - Create reusable exam templates	19
3.2.28	UR2.22 - Access detailed student-level reports	19
3.2.29	UR2.23 - Onboarding aids (tooltips, templates, walkthroughs)	20
3.2.30	UR2.24 - Export exams/question banks/questions per course for archiving	20
3.2.31	UR2.25 - Analyze exam results for suspicious patterns	21
3.3	Functional Requirements Implementation	21
3.3.1	FR1.1 - Secure login (username/password)	21
3.3.2	FR1.2 - Role-based access control (Admin, Instructor)	22
3.3.3	FR1.3 - Password hashing and secure storage	22
3.3.4	FR1.4 - User activity logging	22
3.3.5	FR2.1 - Anonymized student IDs for privacy	23
3.3.6	FR12.1 - Student data anonymization	23
3.3.7	FR12.2 - PII deletion/archiving	24
3.3.8	FR12.3 - Privacy compliance	24
3.3.9	FR3.1 - Question reuse and coverage tracking	25
3.3.10	FR4.1 - Randomized variant generation	25
3.3.11	FR4.2 - PDF export with version codes	25
3.3.12	FR5.1 - Custom marking schemes	26
3.3.13	FR5.2 - Answer key version control	26
3.3.14	FR6.1 - CSV upload for scanned OMR data	27
3.3.15	FR6.2 - Response matching to variants	27
3.3.16	FR6.3 - Validation warnings	28
3.3.17	FR7.1 - Item-level statistics	28
3.3.18	FR7.2 - Data visualizations	28
3.3.19	FR7.3 - Filtered results	29
3.3.20	FR7.4 - Collusion detection	29
3.3.21	FR8.1 - Student report export	30
3.3.22	FR8.2 - Report anonymization	30
3.3.23	FR10.1 - Filtered reports	31
3.3.24	FR9.1 - Shared course editing	31
3.3.25	FR11.1 - Desktop-optimized UI	31
3.3.26	FR11.2 - User guidance	32
3.4	Advanced Features	32
3.4.1	Question substitution recommending system	32
3.4.2	Cover page editing	33
3.4.3	Course/Section Analytics	33
3.4.4	Advanced analytics	34
3.4.5	Partial marking implementation	34
3.4.6	Enhanced Breadcrumb Navigation System	35
3.4.7	Unified Notification System	35
3.4.8	Question Chain	36
3.4.9	UI/UX Component Optimization Framework	36
3.5	Use Cases	37
3.5.1	UC-01: Secure Authentication and Session Management	37
3.5.2	UC-02: Administrator Account Management	38

3.5.3	UC-03: Global System Configuration	38
3.5.4	UC-04: Question Bank Creation and Management	39
3.5.5	UC-05: Course and Section Management	40
3.5.6	UC-06: Exam Creation with Templates	40
3.5.7	UC-07: Variant Generation with Anti-Cheating	41
3.5.8	UC-08: Exam Export and Distribution	42
3.5.9	UC-09: Student-Variant Assignment Tracking	43
3.5.10	UC-10: Answer Key Management	43
3.5.11	UC-11: OMR Data Import and Processing	44
3.5.12	UC-12: Automated Grading and Scoring	45
3.5.13	UC-13: Individual Student Performance Analysis	45
3.5.14	UC-14: Exam-Level Statistical Analysis	46
3.5.15	UC-15: Cross-Exam Comparative Analysis	47
3.5.16	UC-16: Academic Integrity Analysis	48
3.5.17	UC-17: Course and Section Analytics	49
3.5.18	UC-18: System Audit and Monitoring	49
3.5.19	UC-19: Data Privacy and Retention Management	50
3.5.20	UC-20: Collaborative Course Management	51
3.5.21	UC-21: Advanced Question Analysis with ML	51
3.5.22	UC-22: System Onboarding and Training	52
3.5.23	UC-23: Comprehensive Data Export	53
3.6	Use Case Mapping Summary	54
3.6.1	Administrative Use Cases	54
3.6.2	Core Instructor Workflows	54
3.6.3	Supporting Features	54
3.7	Technical Specifications	54
3.7.1	Frontend Service	54
3.7.2	Authentication Service	55
3.7.3	Account Service	56
3.7.4	Exam Generation Service	57
3.7.5	Results Service	58
3.7.6	API Gateway Service	59
3.7.7	BERT Recommendation Service	59
3.7.8	Database Infrastructure	60
4	Status of System Features	60
4.1	Testing Summary	60
4.1.1	Test Type Coverage	60
4.1.2	Test Results	61
4.1.3	Test Summary	61
4.2	Feature status	61
4.2.1	Feature Implementation Overview	61
4.2.2	Implementation Statistics	61
5	UX Analysis	62

6	PM Activities and WBS/FBS	63
6.1	Project Management Strategy	63
6.2	Work Breakdown and Feature Tracking	63
6.3	Effort Estimates and Weekly Contributions	64
6.3.1	Key Performance Metrics	65
6.3.2	Individual Performance Analysis	65
6.4	Work Breakdown Structure (WBS)	65
6.5	Function Breakdown Structure (FBS)	66
6.6	Project Timeline and Delivery Metrics	68
6.6.1	Development Phase Breakdown	68
6.6.2	Final Delivery Summary	69
7	Lesson's Learned and Project Reflections	69
7.1	Dhruv Bhardwaj	69
7.2	Tanner Bjorgan	70
7.3	Qingyun Qian	71
7.4	Peter Szabo	72
7.5	Ziyi Xia	73
7.6	Jasper Xiang	74
7.7	Luke Yin	74
8	Hand-over / Installation / Setup	75
8.1	Contents	75
8.2	System Overview and Service Responsibilities	76
8.3	Environment Configuration and Variables	77
8.4	Run Locally	77
8.5	Production Deployment (Single Host)	78
8.6	tRPC Routing and URLs	78
8.7	Email (SMTP) Configuration	78
8.8	Disabling the BERT Recommendation Service	79
8.9	Course Analytics Cron Job	79
8.10	Nginx Production Config	79
8.11	Docker Production Compose	79
8.12	Seeding	79
8.13	Admin Account Creation	79
8.14	Type Generation for Frontend	80
8.15	RLS Policy Presets and Extending	80
8.16	Exam Status State Machine	80
8.17	Testing (Automated)	80
8.18	User Documentation and Help In-App	80
8.19	Packages and Dependencies	81
9	Conclusion	81

1 Overview

1.1 Motivation

In large post-secondary classrooms, instructors face an escalating challenge: creating fair, secure, and scalable multiple-choice assessments that can withstand academic dishonesty while being easy to administer and analyze. Traditional tools for exam generation often fall short—they lack flexibility, are difficult to use, or fail to provide meaningful analytics after the exam. Moreover, as class sizes grow, the risk of copying between adjacent students increases, especially in offline, optically marked exams.

Our project aims to bridge this gap by providing a modern, full-stack web application that streamlines the creation of randomized exam variants and integrates post-exam analysis into a single workflow. This system empowers instructors with automation, statistical insights, and minimal learning curve, ensuring assessments remain both rigorous and efficient.

1.2 Scope and Boundaries

The project focuses on building a secure, instructor-facing web application for the generation and analysis of randomized multiple-choice question (MCQ) exams. It is scoped to support offline assessment workflows commonly used in large-scale university courses.

The system provides instructors with tools to create question banks, construct and export randomized exam variants, and upload CSV-based student responses for automated grading and analysis. Visual analytics such as score distributions, item difficulty, and variant fairness are built-in to support data-driven reflection and decision-making.

The application supports single-correct MCQs only and does not include live exam-taking functionality or integration with external Learning Management Systems (LMS). Optical mark recognition (OMR) is assumed to be external to the system; the tool begins its workflow from instructor-uploaded CSV data.

From an operational standpoint, the system is designed for desktop browser usage and deployed using containerized infrastructure. Data privacy and integrity are emphasized, aligning with institutional and legal standards.

By clearly limiting the feature set to well-defined workflows and excluding real-time components, the system maintains a lightweight, robust scope that maximizes impact while ensuring feasibility within the capstone timeline.

1.3 Unique Value Proposition

Our system uniquely integrates exam generation and post-exam analytics into a seamless, instructor-friendly experience tailored specifically for large, offline MCQ assessments. While existing platforms may offer some exam construction tools, they often lack the analytical rigor or are too generalized for the challenges of large-scale, optically marked exams. What sets our solution apart is its ability to:

- Automatically generate multiple randomized exam variants to discourage academic dishonesty.
- Provide print-ready PDF exports with embedded version tracking.
- Map student responses back to the correct variant using uploaded CSVs.
- Offer interactive, actionable visualizations for per-question statistics and class performance.
- Flag suspicious patterns indicative of potential collusion or anomalies.
- Perform advanced cheating detection through cross-variant analysis and statistical risk scoring.

Instructors benefit from an intuitive UI that minimizes setup time and eliminates the need for external tooling or scripting. The system prioritizes transparency, fairness, and insight, helping educators make data-driven improvements to their assessments.

1.4 User Groups and Their Needs

The system is designed with two primary user groups in mind: **Instructors** and **Administrators**, each with distinct goals and workflows.

Instructors are faculty members responsible for conducting large-scale offline assessments using multiple-choice questions. Their core needs include:

- Creating and maintaining question banks with metadata (e.g., difficulty, topic).
- Generating randomized exam variants to reduce the likelihood of student collusion.
- Exporting variants as printable PDFs with clear versioning.
- Uploading CSVs of student responses from optically marked sheets.
- Viewing automated grading results and interactive visualizations (e.g., score distribution, per-question difficulty).
- Identifying patterns of academic dishonesty through analytics dashboards and variant-level integrity monitoring.

The system addresses these needs through a seamless, low-effort interface that eliminates the need for manual scripting or spreadsheet manipulation.

Administrators, typically members of the university's IT or teaching support staff, require:

- Secure account creation and management for instructors.
- The ability to reset credentials or assist instructors during technical onboarding.
- Optional oversight of system usage statistics and logs for auditing purposes.

While their interaction with the system is less frequent, administrative tools ensure secure, scalable, and privacy-compliant usage across departments.

2 System Architecture

2.1 Overall Architecture

Our system follows a modular, microservice-inspired architecture designed for scalability, separation of concerns, and maintainability. The application is composed of four backend services - **Authentication**, **Account Management**, **Exam Generation**, and **Results and Analytics** each responsible for a distinct domain of functionality.

These services are accessed via a central tRPC Gateway that exposes a unified API surface to the frontend. The frontend, built with React and Tailwind, communicates exclusively with this gateway, which internally routes requests to the appropriate microservice based on the procedure path.

All services share access to a unified PostgreSQL database. While there is no physical schema separation, logical boundaries are enforced through careful data modeling and application-level constraints. Additionally, **Row-Level Security (RLS)** is implemented in key services such as Exam Generation and Results to ensure that users can only query or modify records they are authorized to access. RLS policies are dynamically applied based on authenticated user identity and role, providing strong data isolation within shared tables.

All components are containerized using Docker, enabling consistent development environments and simplifying deployment. The architecture is designed to support horizontal scaling and could be orchestrated using Docker Compose or Kubernetes in future iterations.

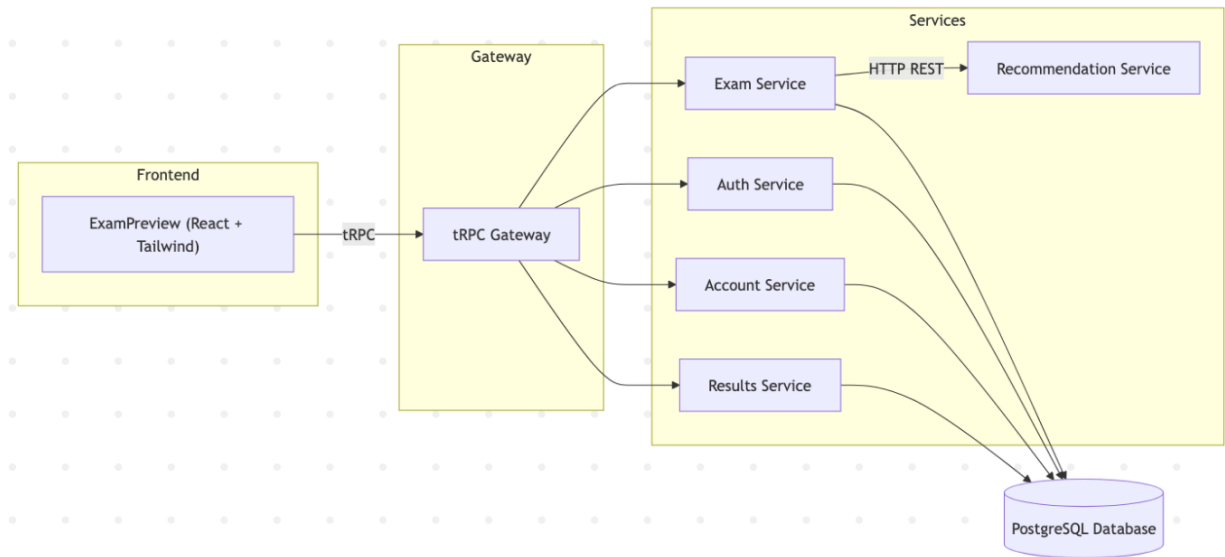


Figure 1: System architecture: the frontend communicates with a tRPC gateway that routes API requests to four independent backend services, all connected to a shared PostgreSQL database.

2.1.1 Key Technical Achievements

- Implemented instructor-aware RLS with dynamic Postgres policies

- Developed snapshot-based schema for preserving exam states (VariantSnapshot, etc.)
- Designed and validated algorithms for variant fairness and cheating detection
- Integrated Prisma with complex joins across snapshots and live entities
- Created scalable service boundaries with unified tRPC gateway

2.1.2 Component Responsibilities

The system is composed of the following core services, each with clear and isolated responsibilities:

- **Authentication Service:** Handles secure login, JWT session management, and token validation for both instructors and administrators. All access control begins here, and downstream services rely on this layer for identity verification.
- **Account Management Service:** Manages user profiles, course-section relationships, and instructor roles. It ensures only authorized users can be linked to specific sessions or courses and provides endpoints for CRUD operations related to account metadata.
- **Exam Generation Service:** Provides tools to manage question banks, generate randomized MCQ variants, and export print-ready PDFs. This service handles the logic for question and option shuffling, variant integrity, and stores mappings for later grading.
- **Result Analytics Service:** Processes uploaded student result CSVs, matches them with correct variants, computes scores, and derives statistical analytics such as score distribution, per-question performance, and variant fairness. It also includes logic to detect suspicious answer patterns and possible collusion using statistical similarity scores. It also includes a cross-variant comparison engine that regrades students on all answer keys and flags anomalous gains. Statistical indicators such as point biserial correlation, score variance, and variant risk levels support academic integrity analysis.

Each service is structured for loose coupling and can be developed and tested independently. This separation of concerns improves maintainability, facilitates testing, and sets the foundation for scalable deployments.

2.1.3 Advanced Security and Data Integrity

The system includes several mechanisms to preserve exam integrity and ensure secure access control:

- **Row-Level Security (RLS):** PostgreSQL enforces strict per-user visibility in exam and result tables
- **JWT-Based Auth:** Session validation via secure tokens
- **Cross-Variant Analysis:** Compares student answers across all variant keys
- **Database Snapshots:** Immutable VariantSnapshot and QuestionSnapshot models preserve exam state for grading
- **Type-Safe API:** Full-stack TypeScript with tRPC ensures backend contracts are validated at compile-time

2.2 Technology Stack

The following technologies were chosen based on their ecosystem maturity, type safety, and ease of integration. Together, they enabled rapid development and reliable deployment across the full stack:

- Frontend: React, TypeScript, Tailwind CSS, Vite
- Backend: Node.js, tRPC (with type-safe procedures), Express.js
- Auth: Custom JWT implementation
- Database: PostgreSQL with Row-Level Security (via Prisma ORM)
- API Gateway: tRPC router with microservice routing
- Containerization: Docker with Docker Compose
- Visualization: Recharts / Chart.js
- PDF Generation: Puppeteer / jsPDF

2.3 Data Flow Diagrams

Level 0 DFD: Overall System Interaction

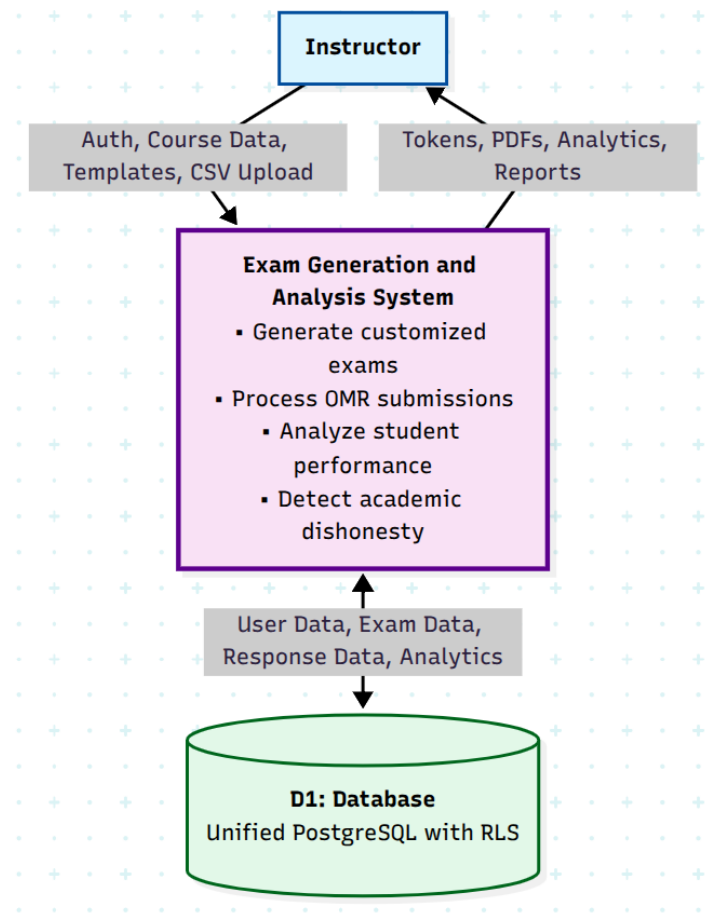


Figure 2: Level 0 DFD: Compact system overview

The Level 0 DFD shows the platform as a single process with one external entity: the Instructor. Instructors authenticate, manage courses and sections, upload exam templates, request variant generation, and upload CSV files containing OMR response data. The system responds with JWT tokens, generated PDF exam variants, performance analytics, and cheating detection reports. All data is stored and retrieved from a unified PostgreSQL database with Row-Level Security (RLS) policies ensuring multi-tenant data isolation.

Level 1 DFD: Microservice Architecture

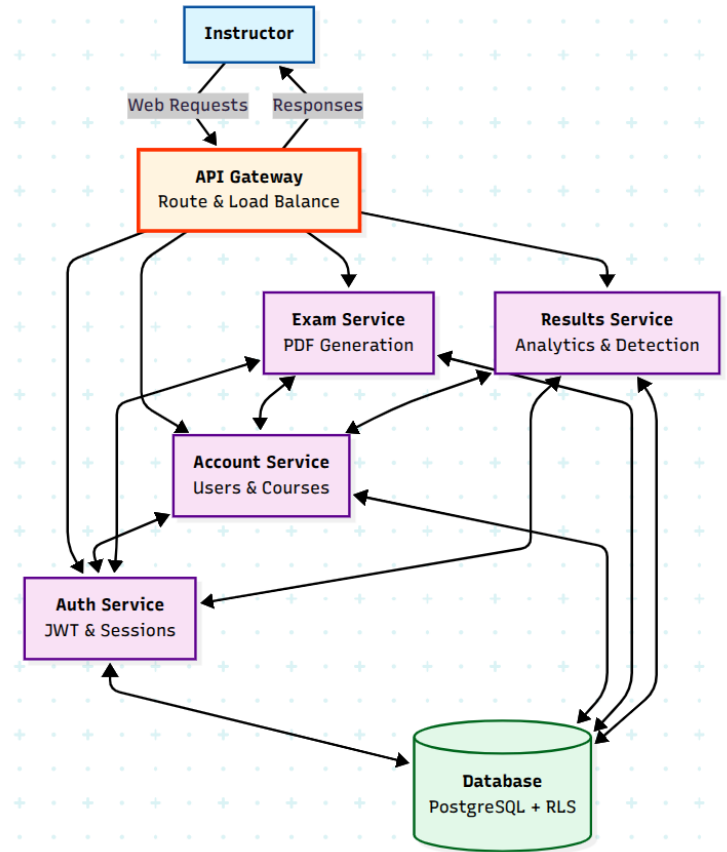


Figure 3: Level 1 DFD: The system decomposes into five microservices coordinated through an API Gateway. Each service handles specific functionality while maintaining secure inter-service communication via tRPC and shared authentication context.

The Level 1 diagram reveals the microservice architecture: the API Gateway routes all external requests and handles CSV file uploads from instructors. The Authentication Service manages JWT-based security and session control. The Account Service handles user profiles and course administration. The Exam Generation Service creates dynamic variants with randomization algorithms. The Results Service processes uploaded CSV data, performs automated grading, and executes sophisticated cheating detection algorithms including cross-variant statistical analysis. All services communicate through tRPC procedures and share access to the unified database with RLS-enforced data isolation.

3 System Features

3.1 Overview of Features

3.2 User Requirements Implementation

3.2.1 UR1.1 - Secure login

Status Complete

Implementation Details

- JWT-based authentication with 24-hour token expiration
- Bcrypt password hashing with salt rounds for secure storage
- Account lockout mechanism after 5 consecutive failed login attempts (10-minute lockout)
- Case-insensitive email matching with Prisma ORM
- HTTP-only cookies with SameSite=Strict for CSRF protection
- Real-time attempt tracking with remaining attempts feedback

Team Member Dhruv Bhardwaj, Tanner Bjorgan, Ziyi Xia, Peter Szabo

Type of Test Integration Testing, Unit Testing, System Functionality Testing

3.2.2 UR1.2 - Instructor account management

Status Complete

Implementation Details

- Complete CRUD operations through dedicated admin endpoints
- Role-based access control with ADMIN/INSTRUCTOR enum validation
- Bulk instructor import via CSV with data validation
- Account activation/deactivation with isActive boolean flag
- Password reset functionality with temporary password expiration
- Audit logging for all account management operations

Team Member Tanner Bjorgan

Type of Test Integration Testing, System Functionality Testing, Unit Testing

3.2.3 UR1.3 - Global settings configuration

Status Complete

Implementation Details

- Global exam template system with customizable layouts
- Marking scheme configuration with per-question weighting
- Cover page template editor with live preview functionality
- System-wide default configurations stored in database
- Template parameter validation and persistence
- Global configuration inheritance for new exams

Team Member Tanner Bjorgan, Jasper Xiang

Type of Test Integration Testing, System Functionality Testing, Unit Testing

3.2.4 UR1.4 - System usage and audit logs

Status Complete

Implementation Details

- Comprehensive audit logging across all microservices
- Real-time activity tracking with PostgreSQL Row-Level Security
- Detailed operation logging including create/update/delete actions
- User activity correlation with actor identification
- Cross-service audit log aggregation and display
- Advanced filtering and search capabilities in admin dashboard

Team Member Peter Szabo, Ziyi Xia

Type of Test Integration Testing, System Functionality Testing, Unit Testing

3.2.5 UR1.5 - Privacy and data retention

Status Complete

Implementation Details

- FIPPA/GDPR compliance with automated data anonymization
- Configurable data retention policies with automated cleanup
- Student ID anonymization for privacy-protected analysis
- PII deletion workflow with audit trail maintenance
- Privacy control toggles for instructors
- Data export restrictions based on privacy settings

Team Member Tanner Bjorgan

Type of Test Integration Testing, System Functionality Testing, Unit Testing

3.2.6 UR2.1 - Secure login with role-based access and password recovery

Status Complete

Implementation Details

- JWT token authentication with role validation middleware
- Email-based password reset with verification code system
- Role-based routing protection in React frontend
- Session management with automatic token refresh
- Failed login attempt tracking per user account
- Secure password storage with bcrypt hashing

Team Member Tanner Bjorgan

Type of Test Integration Testing, System Functionality Testing, Unit Testing

3.2.7 UR2.2 - Build question bank

Status Complete

Implementation Details

- Complete question bank CRUD operations with React forms
- Visibility control system (PUBLIC/PRIVATE states)
- Course association with validation and permission checks

- Creator tracking with ownership-based access control
- Real-time question count tracking and display
- Global state management with custom React hooks

Team Member Jasper Xiang, Ziyi Xia

Type of Test Integration Testing, System Functionality Testing, Unit Testing

3.2.8 UR2.3 - Add questions with correct answers to question bank

Status Complete

Implementation Details

- Rich text editor integration for question content
- Multiple question type support (MULTIPLE_CHOICE, TRUE_FALSE, etc.)
- Answer validation system with correct answer marking
- Metadata tagging with topic/difficulty classification
- Question dependency management with chain creation
- Image upload support with file validation

Team Member Jasper Xiang, Qingyun Qian, Ziyi Xia

Type of Test Unit Testing, Integration Testing

3.2.9 UR2.4 - Import questions from external files with metadata

Status Complete

Implementation Details

- CSV/Excel file parsing with Papa Parse library
- Data validation with error reporting and correction workflow
- Topic/difficulty metadata extraction and assignment
- Batch import processing with progress tracking
- File format validation with user feedback
- Import preview with edit-before-save functionality

Team Member Jasper Xiang, Ziyi Xia

Type of Test Unit Testing, Integration Testing

3.2.10 UR2.5 - Create/manage course

Status Complete

Implementation Details

- Course CRUD operations with section management
- Enrollment capabilities with student roster management
- Department categorization with predefined options
- Course status management (ACTIVE/ARCHIVED)
- Multi-instructor collaboration support
- Course analytics and reporting integration

Team Member Jasper Xiang, Tanner Bjorgan, Ziyi Xia

Type of Test Unit Testing, Integration Testing

3.2.11 UR2.6 - Add questions/question bank to exam

Status Complete

Implementation Details

- Drag-and-drop question selection interface
- Question bank filtering with search functionality
- Validation rules for exam structure requirements
- Block-based organization with position management
- Real-time question preview and editing
- Automatic question allocation for variant generation

Team Member Luke Yin, Qingyun Qian, Jasper Xiang, Ziyi Xia

Type of Test System Functionality Testing, Unit Testing, Integration Testing

3.2.12 UR2.7 - Add/edit/manage exam variants from course

Status Complete

Implementation Details

- Automated variant generation with configurable parameters
- Question shuffling algorithms with dependency awareness
- Option randomization with position tracking
- Variant label assignment (A, B, C, D, E)
- Variant integrity scoring with fairness metrics
- Manual variant editing with real-time preview

Team Member Qingyun Qian, Jasper Xiang, Luke Yin

Type of Test Integration Testing, System Functionality Testing

3.2.13 UR2.8 - Add/modify/manage student rosters for course

Status Complete

Implementation Details

- Student enrollment with bulk CSV import
- Roster management with add/remove functionality
- Student data validation and duplicate detection
- Section assignment with enrollment limits
- Student anonymization for privacy compliance
- Export functionality for roster management

Team Member Jasper Xiang, Ziyi Xia

Type of Test Unit Testing, Integration Testing

3.2.14 UR2.9 - Configure exam parameters

Status Complete

Implementation Details

- Variant count configuration with validation limits
- Questions per exam with block-based organization
- Mandatory question designation and enforcement

- Difficulty distribution controls and validation
- Time limits and exam duration settings
- Scoring configuration with partial credit support

Team Member Luke Yin, Jasper Xiang

Type of Test Integration Testing, System Functionality Testing, Unit Testing

3.2.15 UR2.10(a) - Generate exams with single randomized variant

Status Complete

Implementation Details

- Advanced question shuffling with Fisher-Yates algorithm
- Option randomization preserving answer key accuracy
- Dependency-aware question allocation and ordering
- Statistical analysis for variant fairness validation
- Batch variant generation with progress tracking
- Quality assurance checks for generated variants

Team Member Qingyun Qian, Jasper Xiang, Luke Yin

Type of Test Unit Testing, Integration Testing, System Functionality Testing

3.2.16 UR2.10(b) - Generate exams with multiple randomized variants

Status Complete

Implementation Details

- Multi-variant generation with configurable parameters
- Cross-variant fairness analysis and validation
- Parallel processing for efficient batch generation
- Statistical metrics ensuring variant equality
- Performance optimization for large-scale generation
- Comprehensive quality assurance validation

Team Member Qingyun Qian, Jasper Xiang, Luke Yin

Type of Test Unit Testing, System Functionality Testing

3.2.17 UR2.11 - Export exam docx/PDFs (print-ready)

Status Complete

Implementation Details

- PDF generation with professional formatting
- Unique version codes for each exam variant
- Cover page customization with template system
- Print-ready layout with proper spacing and margins
- Bulk export functionality for multiple variants
- Custom branding and institutional logo integration

Team Member Dhruv Bhardwaj, Luke Yin

Type of Test System Functionality Testing

3.2.18 UR2.12 - Analyze exam statistics for answer similarity per question

Status Complete

Implementation Details

- Question similarity analysis using statistical methods
- Answer pattern analysis with correlation metrics
- Point biserial correlation calculation for item analysis
- Discrimination index computation for question quality
- Cross-variant comparison with fairness assessment
- Statistical visualization with charts and graphs

Team Member Dhruv Bhardwaj, Peter Szabo

Type of Test Integration Testing, Unit Testing, System Functionality Testing, Acceptance Testing

3.2.19 UR2.13 - Track student-variant assignment for answer matching

Status Complete

Implementation Details

- Complete tracking system linking students to variants
- Database relations ensuring answer key accuracy
- Assignment validation with conflict detection
- Historical tracking for re-grading scenarios
- Cross-reference validation for scoring alignment
- Audit trail for assignment changes

Team Member Jasper Xiang

Type of Test Integration Testing, Unit Testing, System Functionality Testing

3.2.20 UR2.14 - Define, edit, and validate answer keys per variant

Status Complete

Implementation Details

- Answer key management interface with validation
- Version control system with immutable snapshots
- Multi-correct answer support for complex questions
- Answer key locking mechanism after finalization
- Validation rules ensuring answer key completeness
- Historical version tracking with rollback capability

Team Member Jasper Xiang

Type of Test Integration Testing, Unit Testing, System Functionality Testing

3.2.21 UR2.15 - Import results from OMR systems

Status Complete

Implementation Details

- CSV upload with bubble sheet data parsing
- OMR format validation with error reporting
- Automatic student-variant matching

- Data integrity checks with validation warnings
- Batch processing with progress indicators
- Error correction workflow with manual override

Team Member Tanner Bjorgan, Dhruv Bhardwaj

Type of Test Integration Testing, System Functionality Testing, Unit Testing

3.2.22 UR2.16 - Track student version assignment for scoring alignment

Status Complete

Implementation Details

- Comprehensive tracking for student-variant pairs
- Scoring alignment validation with answer keys
- Assignment history with change tracking
- Cross-validation between OMR data and assignments
- Integrity checks preventing scoring mismatches
- Audit logging for assignment modifications

Team Member Tanner Bjorgan, Dhruv Bhardwaj

Type of Test Integration Testing, System Functionality Testing, Unit Testing

3.2.23 UR2.17 - Generate student reports with item-level analysis

Status Complete

Implementation Details

- Point biserial correlation for item discrimination
- Score distribution analysis with percentile rankings
- Topic-based performance breakdown and analysis
- Individual student report generation with analytics
- Comprehensive statistics including difficulty analysis
- Export functionality in multiple formats

Team Member Peter Szabo

Type of Test Integration Testing, System Functionality Testing, Unit Testing, Operational Acceptance Testing

3.2.24 UR2.18 - Compare performance across exams and years

Status Complete

Implementation Details

- Cross-exam performance comparison with trend analysis
- Year-over-year performance tracking and visualization
- Student cohort analysis with statistical comparisons
- Historical data correlation and pattern recognition
- Performance metric calculations and benchmarking
- Advanced filtering for targeted comparisons

Team Member Tanner Bjorgan

Type of Test Integration Testing, System Functionality Testing, Unit Testing, Usability Testing

3.2.25 UR2.19 - Export results (CSV, PDF)

Status Complete

Implementation Details

- CSV export with customizable column selection
- PDF report generation with formatted layouts
- Bulk export functionality for class-wide results
- Template-based export with institutional branding
- Data filtering and selection for targeted exports
- Privacy-compliant export with anonymization options

Team Member Tanner Bjorgan, Peter Szabo, Luke Yin

Type of Test Integration Testing, System Functionality Testing, Unit Testing

3.2.26 UR2.20 - Collaborate with co-instructors/TAs within course

Status Complete

Implementation Details

- Shared course editing with concurrent access support
- Activity tracking with real-time collaboration indicators
- Permission-based access control for course resources
- Change history tracking with contributor identification
- Conflict resolution for simultaneous edits
- Notification system for collaborative activities

Team Member Tanner Bjorgan, Ziyi Xia

Type of Test Integration Testing, System Functionality Testing, Operational Acceptance Testing, Unit Testing

3.2.27 UR2.21 - Create reusable exam templates

Status Complete

Implementation Details

- Template system with layout and structure definition
- Section organization with block-based question grouping
- Question selection rules with criteria-based filtering
- Template inheritance and customization capabilities
- Version management with template evolution tracking
- Sharing functionality for institutional template libraries

Team Member Luke Yin

Type of Test Integration Testing, Unit Testing, System Functionality Testing

3.2.28 UR2.22 - Access detailed student-level reports

Status Complete

Implementation Details

- Comprehensive student performance analytics by topic
- Question group analysis with detailed breakdowns
- Individual performance metrics with trend analysis

- Learning outcome correlation and assessment
- Customizable report generation with filtering options
- Export functionality for academic record integration

Team Member Peter Szabo

Type of Test Unit Testing, System Functionality Testing

3.2.29 UR2.23 - Onboarding aids (tooltips, templates, walkthroughs)

Status Complete

Implementation Details

- Interactive step-by-step guided tours with smart positioning
- Contextual tooltips with detailed explanations
- Pre-filled template system for quick start
- Progressive disclosure for complex features
- Database-backed completion tracking across devices
- Responsive design optimized for desktop usage

Team Member Ziyi Xia

Type of Test Integration Testing, Unit Testing, Usability Testing, Acceptance Testing

3.2.30 UR2.24 - Export exams/question banks/questions per course for archiving

Status Complete

Implementation Details

- Comprehensive export for course data archiving
- Question bank export with metadata preservation
- Bulk export functionality with progress tracking
- Format validation ensuring data integrity
- Backup creation with restoration capabilities
- Privacy-compliant export with anonymization

Team Member Jasper Xiang

Type of Test Integration Testing, Unit Testing, System Functionality Testing

3.2.31 UR2.25 - Analyze exam results for suspicious patterns

Status Complete

Implementation Details

- Advanced statistical analysis for cheating detection
- Cross-variant answer pattern comparison
- Similarity scoring with threshold-based flagging
- Machine learning algorithms for pattern recognition
- Detailed reporting with evidence documentation
- Integration with academic integrity workflows

Team Member Dhruv Bhardwaj

Type of Test Integration Testing, Unit Testing, System Functionality Testing

3.3 Functional Requirements Implementation

3.3.1 FR1.1 - Secure login (username/password)

Status Complete

Implementation Details

- JWT-based authentication with 24-hour expiration
- Bcrypt password hashing with configurable salt rounds
- Account lockout after 5 failed attempts (10-minute duration)
- Session management with HTTP-only cookies
- CSRF protection with SameSite=Strict cookie policy
- Real-time login attempt tracking and feedback

Team Member Tanner Bjorgan

Type of Test Integration Testing, Unit Testing, System Functionality Testing, Usability Testing

3.3.2 FR1.2 - Role-based access control (Admin, Instructor)

Status Complete

Implementation Details

- Two-tier role system (ADMIN/INSTRUCTOR) with enum validation
- Middleware-based route protection in Express servers
- Frontend role-based component rendering and navigation
- API endpoint protection with role validation
- Permission inheritance and hierarchical access control
- Real-time role validation with session verification

Team Member Tanner Bjorgan

Type of Test Integration Testing, Unit Testing, System Functionality Testing, Usability Testing

3.3.3 FR1.3 - Password hashing and secure storage

Status Complete

Implementation Details

- Bcrypt hashing with 12 salt rounds for enhanced security
- Secure random password generation for temporary accounts
- Password complexity validation with strength requirements
- Temporary password system with expiration timestamps
- Password reset workflow with email verification
- Historical password tracking preventing reuse

Team Member Tanner Bjorgan

Type of Test Integration Testing, Unit Testing, System Functionality Testing, Usability Testing

3.3.4 FR1.4 - User activity logging

Status Complete

Implementation Details

- Comprehensive audit logging across all microservices
- Real-time activity tracking with PostgreSQL integration
- Structured logging with correlation IDs for request tracking
- Cross-service log aggregation and analysis
- Privacy-compliant logging with data anonymization
- Advanced search and filtering capabilities

Team Member Peter Szabo, Ziyi Xia

Type of Test Integration Testing, Unit Testing

3.3.5 FR2.1 - Anonymized student IDs for privacy

Status Complete

Implementation Details

- Automatic anonymization for statistical analysis
- Configurable privacy settings with instructor override
- FIPPA/GDPR compliance with data protection measures
- Reversible anonymization for authorized access
- Privacy-aware export functionality
- Audit trail for data access and de-anonymization

Team Member Peter Szabo

Type of Test Unit Testing, System Functionality Testing

3.3.6 FR12.1 - Student data anonymization

Status Complete

Implementation Details

- Automatic anonymization for statistical analysis
- Configurable privacy settings with granular control
- FIPPA/GDPR compliance with data protection measures

- Reversible anonymization for authorized access
- Privacy-aware export functionality
- Audit trail for data access and modifications

Team Member Peter Szabo

Type of Test Unit Testing, System Functionality Testing

3.3.7 FR12.2 - PII deletion/archiving

Status Complete

Implementation Details

- Automated data retention policies with configurable timelines
- Secure deletion with cryptographic verification
- Archival functionality with long-term storage options
- Compliance tracking for regulatory requirements
- Audit trail preservation for accountability
- Data recovery procedures for emergency situations

Team Member Tanner Bjorgan

Type of Test Integration Testing, Unit Testing

3.3.8 FR12.3 - Privacy compliance

Status Complete

Implementation Details

- Comprehensive FIPPA/GDPR compliance framework
- Privacy control toggles with user-friendly interface
- Data protection impact assessments
- Regular compliance audits with automated reporting
- Privacy policy integration with system notifications
- Legal compliance tracking and documentation

Team Member Tanner Bjorgan

Type of Test Integration Testing, Unit Testing

3.3.9 FR3.1 - Question reuse and coverage tracking

Status Complete

Implementation Details

- Question usage frequency tracking across exams
- Coverage analysis with visual indicators
- Reuse prevention algorithms for exam variety
- Question pool management with rotation strategies
- Metadata-based tracking and categorization
- Historical usage reports and analytics

Team Member Qingyun Qian, Jasper Xiang

Type of Test Integration Testing, Unit Testing, System Functionality Testing, Usability Testing

3.3.10 FR4.1 - Randomized variant generation

Status Complete

Implementation Details

- Fisher-Yates shuffling algorithm for question randomization
- Constraint-based substitution from question pools
- Dependency-aware question ordering and validation
- Statistical fairness analysis for variant equality
- Configurable randomization parameters
- Quality assurance validation for generated variants

Team Member Qingyun Qian

Type of Test Integration Testing, Unit Testing, System Functionality Testing, Usability Testing

3.3.11 FR4.2 - PDF export with version codes

Status Complete

Implementation Details

- Professional PDF formatting with institutional branding
- Unique QR codes and version identifiers per variant
- Print-ready layout optimization for standard paper sizes
- Batch export functionality with progress tracking
- Custom cover page integration with template system
- Accessibility features for inclusive design

Team Member Jasper Xiang, Luke Yin

Type of Test Integration Testing, Unit Testing, System Functionality Testing, Usability Testing

3.3.12 FR5.1 - Custom marking schemes

Status Complete

Implementation Details

- Per-question point allocation with decimal precision
- Negative marking support for incorrect answers
- Multi-correct answer framework with partial credit
- Flexible scoring algorithms with custom weighting
- Template-based marking scheme inheritance
- Real-time score calculation and validation

Team Member Jasper Xiang

Type of Test Unit Testing, System Functionality Testing, Usability Testing

3.3.13 FR5.2 - Answer key version control

Status Complete

Implementation Details

- Immutable snapshot creation upon finalization
- Version tracking with change history documentation
- Answer key locking mechanism preventing unauthorized changes

- Rollback functionality for error correction
- Audit trail for all answer key modifications
- Cross-variant consistency validation

Team Member Jasper Xiang

Type of Test Unit Testing, System Functionality Testing, Usability Testing

3.3.14 FR6.1 - CSV upload for scanned OMR data

Status Complete

Implementation Details

- Papa Parse integration for robust CSV processing
- File validation with format checking and error reporting
- Batch processing with progress indicators
- Data sanitization and format standardization
- Error correction workflow with manual override
- Integration with bubble sheet scanning systems

Team Member Dhruv Bhardwaj, Tanner Bjorgan

Type of Test Integration Testing, Unit Testing, System Functionality Testing, Usability Testing

3.3.15 FR6.2 - Response matching to variants

Status Complete

Implementation Details

- Automatic student-variant assignment validation
- Answer key correlation with response data
- Mismatch detection with warning notifications
- Cross-reference validation ensuring scoring accuracy
- Historical tracking for re-grading scenarios
- Integrity checks preventing data corruption

Team Member Dhruv Bhardwaj, Tanner Bjorgan

Type of Test Integration Testing, Unit Testing, System Functionality Testing

3.3.16 FR6.3 - Validation warnings

Status Complete

Implementation Details

- Comprehensive scan completeness validation
- Student ID recognition with fuzzy matching
- Version mismatch detection and reporting
- Data integrity warnings with corrective suggestions
- Real-time validation feedback during upload
- Detailed error reporting with actionable recommendations

Team Member Dhruv Bhardwaj, Tanner Bjorgan

Type of Test Integration Testing, Unit Testing, System Functionality Testing

3.3.17 FR7.1 - Item-level statistics

Status Complete

Implementation Details

- Point biserial correlation calculation for discrimination
- Item difficulty analysis with distribution metrics
- Response pattern analysis with visual representation
- Statistical significance testing for question validity
- Comprehensive metrics dashboard with real-time updates
- Export functionality for statistical software integration

Team Member Tanner Bjorgan, Peter Szabo

Type of Test System Functionality Testing, Unit Testing

3.3.18 FR7.2 - Data visualizations

Status Complete

Implementation Details

- Interactive score distribution histograms
- Difficulty-discrimination scatter plots with clustering
- Year-over-year performance trend analysis
- Question-level analytics with visual indicators
- Responsive chart design with mobile optimization
- Export functionality for presentation materials

Team Member Tanner Bjorgan, Peter Szabo

Type of Test System Functionality Testing, Unit Testing

3.3.19 FR7.3 - Filtered results

Status Complete

Implementation Details

- Multi-dimensional filtering with advanced criteria
- Real-time filter application with instant results
- Saved filter configurations for repeated analysis
- Export functionality preserving filter settings
- Performance optimization for large datasets
- User-friendly filter interface with intuitive controls

Team Member Tanner Bjorgan, Ziyi Xia, Peter Szabo, Luke Yin

Type of Test Integration Testing, Unit Testing, System Functionality Testing, Usability Testing

3.3.20 FR7.4 - Collusion detection

Status Complete

Implementation Details

- Statistical pattern analysis for answer similarity
- Cross-variant cheating detection algorithms
- Threshold-based flagging with confidence scoring

- Detailed reporting with evidence documentation
- Interactive visualization of suspicious patterns
- Integration with academic integrity reporting systems

Team Member Dhruv Bhardwaj

Type of Test Unit Testing, System Functionality Testing

3.3.21 FR8.1 - Student report export

Status Complete

Implementation Details

- Individual and bulk PDF report generation
- CSV export with customizable data selection
- Template-based report formatting with branding
- Automated report distribution via email
- Performance analytics integration with detailed metrics
- Privacy-compliant export with anonymization options

Team Member Peter Szabo

Type of Test System Functionality Testing, Unit Testing

3.3.22 FR8.2 - Report anonymization

Status Complete

Implementation Details

- Configurable anonymization levels with instructor control
- Privacy-aware data export with selective anonymization
- Reversible anonymization for authorized access
- Compliance tracking for data protection regulations
- Audit trail for anonymization and de-anonymization
- Template-based anonymization rules

Team Member Peter Szabo

Type of Test System Functionality Testing, Unit Testing

3.3.23 FR10.1 - Filtered reports

Status Complete

Implementation Details

- Advanced filtering with multiple criteria support
- Saved filter configurations for repeated use
- Real-time report generation with progress tracking
- Export functionality in multiple formats
- Performance optimization for large datasets
- User-friendly interface with intuitive controls

Team Member Tanner Bjorgan, Ziyi Xia, Peter Szabo

Type of Test Integration Testing, Unit Testing, System Functionality Testing

3.3.24 FR9.1 - Shared course editing

Status Complete

Implementation Details

- Real-time collaboration with conflict resolution
- Activity tracking with contributor identification
- Permission-based access control for course resources
- Change history with detailed modification logs
- Notification system for collaborative activities
- Version control for course materials and exams

Team Member Ziyi Xia

Type of Test Integration Testing, Unit Testing

3.3.25 FR11.1 - Desktop-optimized UI

Status Complete

Implementation Details

- Responsive design optimized for desktop workflows
- Modern React component architecture with TypeScript
- Accessibility features with ARIA compliance
- Performance optimization with lazy loading
- Consistent design system with UBC branding
- Cross-browser compatibility testing and validation

Team Member Ziyi Xia

Type of Test Unit Testing, Integration Testing, System Functionality Testing

3.3.26 FR11.2 - User guidance

Status Complete

Implementation Details

- Interactive onboarding tours with contextual help
- Pre-filled templates for quick feature adoption
- Comprehensive tooltip system with detailed explanations
- Progressive disclosure for complex features
- Video tutorials and documentation integration
- User feedback collection and interface improvement

Team Member Ziyi Xia

Type of Test Unit Testing, Integration Testing, System Functionality Testing

3.4 Advanced Features

3.4.1 Question substitution recommending system

Status Complete

Motivation Making meaningful question substitution across variants could be difficult. With the help of local machine learning model, questions can be easily substituted.

Implementation Details

- BERT embeddings for semantic similarity analysis
- Machine learning algorithms for question recommendation
- Duplicate detection across question repositories
- Contextual substitution suggestions with similarity scoring
- Integration with exam generation workflow
- Performance optimization for real-time recommendations

Team Member Qingyun Qian

Type of Test Integration Testing, Unit Testing, System Functionality Testing

3.4.2 Cover page editing

Status Complete

Motivation To make the exported exam sheet ready to print, we want to allow the instructor to edit the cover page with customized templates to simplify the procedure.

Implementation Details

- Personal cover page editor with live preview
- Template parameter system with customizable fields
- Persistence via global and personal configuration storage
- Per-exam assignment with template inheritance
- Rich text editing with formatting options
- Integration with PDF export functionality

Team Member Luke Yin, Tanner Bjorgan

Type of Test Integration Testing, Unit Testing, System Functionality Testing

3.4.3 Course/Section Analytics

Status Complete

Motivation Beyond the required course analytics, since the uniqueness of our structure, we can do course wise comparison between sections.

Implementation Details

- Aggregated course and section performance metrics
- Item-level statistics with comprehensive analysis
- Score distribution analysis with visual representation
- Performance trend tracking across time periods
- Interactive dashboards with real-time data updates
- Downloadable reports with customizable formats

Team Member Tanner Bjorgan, Ziyi Xia

Type of Test Integration Testing, System Functionality Testing, Unit Testing

3.4.4 Advanced analytics

Status Complete

Implementation Details

- Multi-level analytics (exam, question, student)
- Interactive histogram visualizations
- Difficulty-discrimination scatter plot analysis
- Performance trend analysis with historical comparison
- Flexible filtering system with advanced criteria
- Export functionality for external analysis tools

Team Member Peter Szabo, Luke Yin, Ziyi Xia

Type of Test System Functionality Testing, Unit Testing

3.4.5 Partial marking implementation

Status Complete

Motivation For future support of multiple correct questions, the database and grading engine have been prepared.

Implementation Details

- Database schema supporting partial credit scoring for multiple correct questions
- Negative marking implementation in grading algorithms
- Multi-correct question partial credit calculation
- Flexible scoring rules with configurable weights
- Real-time score calculation and validation
- Integration with marking scheme templates

Team Member Jasper Xiang

Type of Test Unit Testing, System Functionality Testing

3.4.6 Enhanced Breadcrumb Navigation System

Status Complete

Implementation Details

- Dynamic breadcrumb generation based on navigation context
- URL parameter parsing for contextual information display
- Smart breadcrumb hierarchy with intelligent truncation
- Real-time updates reflecting current application state
- Integration with React Router for seamless navigation
- Accessibility features with screen reader support

Team Member Ziyi Xia

Type of Test Integration Testing, Unit Testing, Usability Testing

3.4.7 Unified Notification System

Status Complete

Implementation Details

- Application-wide notification architecture with centralized management
- Deduplication mechanism preventing identical notifications
- Multiple notification types with customizable styling

- Configurable duration settings with user preferences
- Toast notification system with queue management
- Integration with loading states and error handling

Team Member Ziyi Xia

Type of Test Integration Testing, Unit Testing, System Functionality Testing

3.4.8 Question Chain

Status Complete

Implementation Details

- Visual question dependency management with drag-and-drop
- Chain editing interface with real-time validation
- Dependency conflict detection and resolution
- Interactive visualization of question relationships
- Bulk chain operations with progress tracking
- Integration with backend dependency validation

Team Member Ziyi Xia, Qingyun Qian

Type of Test Integration Testing, Unit Testing, System Functionality Testing

3.4.9 UI/UX Component Optimization Framework

Status Complete

Implementation Details

- Comprehensive tooltip system with smart positioning
- Responsive design improvements across all components
- Interaction feedback enhancement with visual cues
- Performance optimization with memoization strategies
- Accessibility improvements with ARIA label compliance
- Cross-device compatibility testing and validation

Team Member Ziyi Xia

Type of Test Unit Testing, Integration Testing, Usability Testing

The system exceeds the original requirements with additional enterprise-grade features including advanced analytics, BERT-based question analysis, cover page editing, course/section analytics, partial marking capabilities, and comprehensive UI/UX enhancements.

3.5 Use Cases

This document presents the comprehensive use cases for the Exam Generation and Analytics System, organized by actor and functional area. Each use case is mapped to specific requirements from the feature set.

3.5.1 UC-01: Secure Authentication and Session Management

Primary Actor: All Users (Admin/Instructor)

Goal: Authenticate users securely and manage login sessions with appropriate security measures

Preconditions: User has registered account in the system

Postconditions: User is authenticated with role-appropriate access and session established

Main Success Scenario:

1. User accesses the system login page
2. System displays authentication form with attempt counter
3. User enters email (case-insensitive) and password
4. System validates credentials using bcrypt comparison
5. System checks account lockout status (5 attempts = 10-min lockout)
6. System generates JWT token (24-hour expiration)
7. System establishes HTTP-only cookie with CSRF protection
8. System redirects to role-appropriate dashboard
9. System tracks login activity in audit log

Extensions:

- 4a. Invalid credentials: Display error with remaining attempts
- 5a. Account locked: Show lockout timer and message
- 6a. Password expired: Redirect to password reset flow
- 7a. Previous session exists: Invalidate old session

Related Requirements: UR1.1, UR2.1, FR1.1, FR1.2, FR1.3

3.5.2 UC-02: Administrator Account Management

Primary Actor: Administrator

Goal: Manage instructor accounts with complete lifecycle control

Preconditions: User has ADMIN role privileges

Postconditions: Instructor accounts created/modified with proper audit trail

Main Success Scenario:

1. Admin accesses instructor management interface
2. System displays current instructor roster with status
3. Admin performs account operation (create/edit/deactivate)
4. For bulk operations, admin uploads CSV file
5. System validates data and checks for duplicates
6. System creates accounts with temporary passwords
7. System sends activation emails to new instructors
8. System logs all operations in audit trail
9. Admin can view account activity history

Extensions:

- 4a. CSV format error: Display validation report
- 5a. Duplicate email: Prompt for override or skip
- 6a. Password reset: Generate temporary password with expiration
- 8a. Suspicious activity: Flag in audit log

Related Requirements: UR1.2, FR1.2, FR1.4

3.5.3 UC-03: Global System Configuration

Primary Actor: Administrator

Goal: Configure system-wide settings and templates

Preconditions: User has ADMIN role

Postconditions: Global configurations applied across all services

Main Success Scenario:

1. Admin accesses global settings interface
2. System displays current configuration parameters
3. Admin configures exam template defaults
4. Admin sets up marking scheme templates

5. Admin customizes cover page templates with live preview
6. Admin establishes data retention policies
7. System validates and persists configurations
8. System propagates settings to all microservices
9. New exams inherit global defaults

Extensions:

- 5a. Template editor: Use rich text with parameter placeholders
- 6a. Privacy settings: Configure FIPPA/GDPR compliance rules
- 7a. Validation error: Highlight invalid parameters

Related Requirements: UR1.3, UR1.5, FR12.2, FR12.3

3.5.4 UC-04: Question Bank Creation and Management

Primary Actor: Instructor

Goal: Build comprehensive question repositories with metadata

Preconditions: Instructor has course access

Postconditions: Question bank available for exam creation

Main Success Scenario:

1. Instructor accesses question bank interface
2. System displays existing banks with visibility status
3. Instructor creates new bank with course association
4. Instructor adds questions with rich text editor
5. System supports multiple question types (MC, TF, etc.)
6. Instructor tags questions with topic/difficulty metadata
7. Instructor sets visibility (PUBLIC/PRIVATE)
8. System validates and stores questions
9. System tracks creator and usage statistics

Extensions:

- 4a. Bulk import: Parse CSV/Excel with validation
- 5a. Image upload: Validate file type and size
- 6a. Question chains: Define dependencies between questions
- 8a. Duplicate detection: Use BERT similarity analysis

Related Requirements: UR2.2, UR2.3, UR2.4, FR3.1

3.5.5 UC-05: Course and Section Management

Primary Actor: Instructor

Goal: Create and manage courses with sections and enrollment

Preconditions: Instructor account is active

Postconditions: Course structure established with enrolled students

Main Success Scenario:

1. Instructor navigates to course management
2. System displays instructor's courses
3. Instructor creates course with details (code, name, term)
4. Instructor adds sections with enrollment limits
5. Instructor imports student roster via CSV
6. System validates student data and detects duplicates
7. Instructor assigns students to sections
8. System maintains enrollment history
9. Instructor can add co-instructors for collaboration

Extensions:

- 5a. Manual entry: Add students individually
- 6a. Data validation: Report format errors
- 7a. Section full: Suggest alternative sections
- 9a. Permissions: Set access levels for collaborators

Related Requirements: UR2.5, UR2.8, UR2.20, FR9.1

3.5.6 UC-06: Exam Creation with Templates

Primary Actor: Instructor

Goal: Design comprehensive exams using question banks

Preconditions: Question banks exist with sufficient questions

Postconditions: Exam created and ready for variant generation

Main Success Scenario:

1. Instructor initiates exam creation wizard
2. System presents template options or blank start
3. Instructor configures exam parameters:

- Variant count and randomization rules
 - Questions per exam and block organization
 - Mandatory question designation
 - Difficulty distribution requirements
 - Time limits and scoring configuration
4. Instructor selects questions from banks
 5. System validates question allocation
 6. Instructor arranges questions in blocks
 7. System creates exam in DRAFT status
 8. Instructor can preview exam structure

Extensions:

- 2a. Template use: Apply pre-built exam structure
- 4a. Drag-drop interface: Visual question selection
- 5a. Insufficient questions: Suggest alternatives
- 6a. Dependencies: Maintain question chains

Related Requirements: UR2.6, UR2.9, UR2.21, FR5.1

3.5.7 UC-07: Variant Generation with Anti-Cheating

Primary Actor: Instructor

Goal: Generate multiple exam variants with optimal differentiation

Preconditions: Exam exists in DRAFT with configured parameters

Postconditions: Variants created with fairness validation

Main Success Scenario:

1. Instructor selects exam for variant generation
2. System analyzes question pool and dependencies
3. Instructor confirms variant parameters
4. System generates variants using:
 - Fisher-Yates shuffling for questions
 - Option randomization with answer key tracking
 - Dependency-aware allocation
5. System performs fairness analysis

6. System calculates cross-variant similarity metrics
7. Instructor reviews variant preview
8. System finalizes variants with unique codes

Extensions:

- 4a. Single variant: Generate one randomized version
- 4b. Multiple variants: Create configured number
- 5a. Unfair distribution: Alert and suggest fixes
- 6a. High similarity: Recommend more questions

Related Requirements: UR2.7, UR2.10(a), UR2.10(b), FR4.1

3.5.8 UC-08: Exam Export and Distribution

Primary Actor: Instructor

Goal: Export print-ready exam documents

Preconditions: Exam variants are finalized

Postconditions: PDF/DOCX files ready for printing

Main Success Scenario:

1. Instructor accesses exam export interface
2. System displays available variants
3. Instructor customizes cover page template
4. Instructor selects export format (PDF/DOCX)
5. System generates documents with:
 - Unique version codes per variant
 - Professional formatting and margins
 - Custom branding/logos
 - QR codes for tracking
6. Instructor can bulk export all variants
7. System provides download links

Extensions:

- 3a. Template editor: Live preview of changes
- 5a. Accessibility: Include screen reader tags
- 6a. Large batch: Show progress indicator

Related Requirements: UR2.11, FR4.2

3.5.9 UC-09: Student-Variant Assignment Tracking

Primary Actor: Instructor

Goal: Track which students receive which exam variants

Preconditions: Variants generated and students enrolled

Postconditions: Complete mapping for accurate grading

Main Success Scenario:

1. Instructor accesses assignment interface
2. System displays student roster and variants
3. Instructor assigns variants to students
4. System validates no conflicts or duplicates
5. System maintains assignment history
6. Instructor can modify assignments pre-exam
7. System locks assignments after exam start
8. System creates audit trail of changes

Extensions:

- 3a. Random assignment: System auto-assigns
- 3b. Manual override: Specific assignments
- 5a. Re-assignment: Track version history

Related Requirements: UR2.13, UR2.16

3.5.10 UC-10: Answer Key Management

Primary Actor: Instructor

Goal: Define and validate answer keys per variant

Preconditions: Exam variants exist

Postconditions: Answer keys finalized and version-controlled

Main Success Scenario:

1. Instructor accesses answer key interface
2. System displays variants with current keys
3. Instructor defines correct answers per question
4. System supports multi-correct answers
5. Instructor configures partial credit rules

6. System validates key completeness
7. Instructor finalizes answer keys
8. System creates immutable snapshot
9. System locks keys preventing changes

Extensions:

- 4a. Negative marking: Set penalty scores
- 5a. Partial credit: Define scoring rules
- 8a. Version control: Track all changes
- 9a. Emergency unlock: Admin override only

Related Requirements: UR2.14, FR5.1, FR5.2

3.5.11 UC-11: OMR Data Import and Processing

Primary Actor: Instructor

Goal: Import and process bubble sheet scan results

Preconditions: Answer keys defined, students assigned to variants

Postconditions: Student responses imported and validated

Main Success Scenario:

1. Instructor accesses OMR import interface
2. System displays upload requirements
3. Instructor uploads CSV from bubble sheet scanner
4. System parses file using Papa Parse
5. System validates format and data integrity
6. System matches students to assigned variants
7. System detects and reports mismatches
8. Instructor reviews validation warnings
9. System processes and stores responses

Extensions:

- 5a. Format error: Display specific issues
- 6a. Unknown student: Flag for review
- 7a. Version mismatch: Alert instructor
- 8a. Manual correction: Edit before import

Related Requirements: UR2.15, FR6.1, FR6.2, FR6.3

3.5.12 UC-12: Automated Grading and Scoring

Primary Actor: System (triggered by Instructor)

Goal: Calculate scores based on answer keys and marking schemes

Preconditions: Student responses imported, answer keys defined

Postconditions: Grades calculated with detailed breakdowns

Main Success Scenario:

1. System retrieves student responses
2. System loads appropriate variant answer keys
3. System applies marking scheme:
 - Per-question point allocation
 - Negative marking if configured
 - Partial credit for multi-correct
4. System calculates individual scores
5. System generates score breakdowns by:
 - Question
 - Topic
 - Difficulty level
6. System stores results in database
7. System triggers analytics calculations

Extensions:

- 3a. Custom scheme: Apply special rules
- 4a. Bonus questions: Add extra points
- 5a. Missing responses: Handle as configured

Related Requirements: FR5.1, FR5.2

3.5.13 UC-13: Individual Student Performance Analysis

Primary Actor: Instructor

Goal: Generate detailed performance reports for students

Preconditions: Exam graded with results available

Postconditions: Comprehensive student reports generated

Main Success Scenario:

1. Instructor accesses student reports section
2. System displays graded exam list
3. Instructor selects exam and students
4. System generates individual reports with:
 - Overall score and percentile
 - Question-by-question analysis
 - Topic performance breakdown
 - Comparison to class average
5. Instructor configures privacy settings
6. System applies anonymization if required
7. Instructor exports reports (PDF/CSV)
8. System can email reports to students

Extensions:

- 5a. FIPPA compliance: Auto-anonymize
- 6a. Selective hiding: Remove identifiers
- 7a. Bulk export: Generate all reports

Related Requirements: UR2.17, UR2.22, FR2.1, FR8.1, FR8.2

3.5.14 UC-14: Exam-Level Statistical Analysis

Primary Actor: Instructor

Goal: Analyze overall exam performance and question quality

Preconditions: Sufficient students completed exam

Postconditions: Statistical insights available for improvement

Main Success Scenario:

1. Instructor accesses analytics dashboard
2. System displays exam selection interface
3. Instructor chooses exam and analysis type
4. System calculates comprehensive metrics:
 - Score distribution (mean, median, std dev)
 - Item difficulty analysis
 - Point biserial correlation

- Discrimination indices
- 5. System generates visualizations:
 - Histograms for score distribution
 - Scatter plots for item analysis
 - Heat maps for topic performance
- 6. Instructor applies filters for deeper analysis
- 7. System provides actionable insights

Extensions:

- 4a. Small sample: Warning about reliability
- 5a. Interactive charts: Drill-down capability
- 6a. Multi-criteria: Complex filtering
- 7a. Recommendations: Suggest improvements

Related Requirements: UR2.12, FR7.1, FR7.2, FR7.3

3.5.15 UC-15: Cross-Exam Comparative Analysis

Primary Actor: Instructor

Goal: Compare performance across multiple exams/terms

Preconditions: Multiple exams with results exist

Postconditions: Comparative insights and trends identified

Main Success Scenario:

1. Instructor accesses comparative analytics
2. System displays available exams by term/year
3. Instructor selects exams for comparison
4. System performs cross-exam analysis:
 - Performance trends over time
 - Topic consistency analysis
 - Difficulty progression
 - Student cohort comparisons
5. System generates trend visualizations
6. Instructor can drill into specific metrics
7. System identifies significant changes

8. Instructor exports comparison reports

Extensions:

- 4a. Different structures: Normalize for comparison
- 5a. Missing data: Indicate gaps in timeline
- 7a. Anomaly detection: Flag unusual patterns

Related Requirements: UR2.18, FR7.2, FR7.3

3.5.16 UC-16: Academic Integrity Analysis

Primary Actor: Instructor

Goal: Detect potential cheating through statistical analysis

Preconditions: Multiple students completed same exam

Postconditions: Suspicious patterns identified and documented

Main Success Scenario:

1. Instructor initiates integrity analysis
2. System analyzes all student responses
3. System performs dual-algorithm detection:
 - Answer similarity patterns
 - Cross-variant answer sharing
 - Statistical significance testing
4. System calculates trust scores
5. System flags suspicious pairs/groups
6. System generates detailed evidence report
7. Instructor reviews flagged cases
8. System maintains investigation audit trail

Extensions:

- 3a. Insufficient data: Minimum 10 students required
- 5a. High confidence: Priority flagging
- 6a. Visual comparison: Side-by-side answers
- 8a. Case notes: Document follow-up actions

Related Requirements: UR2.25, FR7.4

3.5.17 UC-17: Course and Section Analytics

Primary Actor: Instructor

Goal: Analyze performance at course/section level

Preconditions: Course has completed exams with results

Postconditions: Aggregated analytics available for course improvement

Main Success Scenario:

1. Instructor accesses course analytics
2. System displays course/section selector
3. Instructor chooses analysis scope
4. System aggregates performance data:
 - Section-wise comparisons
 - Topic mastery across sections
 - Instructor effectiveness metrics
 - Learning outcome achievement
5. System generates dashboards
6. Instructor can compare sections
7. System provides downloadable reports

Extensions:

- 4a. Multiple instructors: Compare teaching impact
- 5a. Real-time updates: Live data refresh
- 6a. Statistical tests: Significance of differences

Related Requirements: Advanced Feature - Course/Section Analytics

3.5.18 UC-18: System Audit and Monitoring

Primary Actor: Administrator

Goal: Monitor system usage and maintain audit trail

Preconditions: System operational with activity

Postconditions: Complete audit trail with compliance tracking

Main Success Scenario:

1. Admin accesses audit dashboard
2. System displays activity timeline
3. Admin applies filters (user, action, date)

4. System shows detailed operation logs:
 - User actions with timestamps
 - Data modifications with before/after
 - Cross-service activity correlation
 - Login attempts and security events
5. Admin searches for specific events
6. System generates compliance reports
7. Admin exports audit logs

Extensions:

- 4a. Security alerts: Highlight suspicious activity
- 5a. Advanced search: Complex query builder
- 6a. Retention policy: Auto-archive old logs

Related Requirements: UR1.4, FR1.4

3.5.19 UC-19: Data Privacy and Retention Management

Primary Actor: Administrator

Goal: Ensure compliance with privacy regulations

Preconditions: System contains student data

Postconditions: Data handled according to privacy laws

Main Success Scenario:

1. Admin accesses privacy management interface
2. System displays current privacy settings
3. Admin configures retention policies:
 - Automatic anonymization schedules
 - Data deletion timelines
 - Archive strategies
4. System validates against FIPPA/GDPR
5. Admin sets privacy control toggles
6. System applies policies automatically
7. System maintains compliance audit trail
8. Admin can generate compliance reports

Extensions:

- 3a. Manual trigger: Immediate anonymization
- 4a. Compliance check: Warn about violations
- 6a. Selective application: Per-course settings

Related Requirements: UR1.5, FR2.1, FR12.1, FR12.2, FR12.3

3.5.20 UC-20: Collaborative Course Management

Primary Actor: Multiple Instructors/TAs

Goal: Enable team-based course administration

Preconditions: Course has multiple authorized users

Postconditions: Changes synchronized across all users

Main Success Scenario:

1. Primary instructor adds collaborators
2. System assigns appropriate permissions
3. Collaborators access shared course
4. System tracks all user activities
5. Multiple users can edit simultaneously
6. System handles conflict resolution
7. Changes reflect in real-time
8. System maintains change history
9. Users receive activity notifications

Extensions:

- 2a. Role-based: Different permission levels
- 5a. Lock mechanism: Prevent conflicts
- 6a. Merge conflicts: Manual resolution
- 9a. Configurable: Notification preferences

Related Requirements: UR2.20, FR9.1

3.5.21 UC-21: Advanced Question Analysis with ML

Primary Actor: Instructor

Goal: Use machine learning for question quality and substitution

Preconditions: Question bank with sufficient questions

Postconditions: ML-powered insights and recommendations

Main Success Scenario:

1. Instructor accesses question analysis
2. System loads BERT embedding model
3. Instructor selects questions for analysis
4. System performs semantic similarity analysis
5. System detects near-duplicates
6. System recommends substitutions
7. Instructor reviews suggestions
8. System learns from instructor choices

Extensions:

- 4a. First run: Generate embeddings
- 5a. Threshold config: Adjust similarity level
- 6a. Context-aware: Consider topic/difficulty

Related Requirements: Advanced Feature - Question substitution recommending system

3.5.22 UC-22: System Onboarding and Training

Primary Actor: New Instructor

Goal: Learn system features through guided experience

Preconditions: User has new account

Postconditions: User proficient with core features

Main Success Scenario:

1. New user logs in for first time
2. System initiates interactive tour
3. User follows step-by-step guide
4. System provides contextual tooltips
5. User accesses pre-filled templates
6. System tracks completion progress
7. User can replay tutorials
8. System offers advanced tips progressively

Extensions:

- 3a. Skip option: Experienced users

- 5a. Template library: Multiple examples
- 7a. Video tutorials: Embedded guides
- 8a. Feature discovery: Highlight new features

Related Requirements: UR2.23, FR11.2

3.5.23 UC-23: Comprehensive Data Export

Primary Actor: Instructor/Administrator

Goal: Export system data for archiving or migration

Preconditions: User has appropriate permissions

Postconditions: Data exported in portable format

Main Success Scenario:

1. User accesses export interface
2. System displays exportable entities
3. User selects data scope:
 - Exams and variants
 - Question banks
 - Student results
 - Analytics reports
4. User chooses export format
5. System validates privacy settings
6. System generates export package
7. User downloads archive
8. System logs export activity

Extensions:

- 5a. Anonymization: Apply privacy rules
- 6a. Large export: Background processing
- 7a. Chunked download: Multiple files

Related Requirements: UR2.19, UR2.24

3.6 Use Case Mapping Summary

3.6.1 Administrative Use Cases

- **UC-01, UC-02, UC-03, UC-18, UC-19:** Cover all admin requirements (UR1.x series)

3.6.2 Core Instructor Workflows

- **UC-04 to UC-10:** Cover exam creation workflow (UR2.1-2.14)
- **UC-11, UC-12:** Cover grading workflow (UR2.15-2.16)
- **UC-13 to UC-17:** Cover analytics workflow (UR2.17-2.18, 2.22, 2.25)

3.6.3 Supporting Features

- **UC-20:** Collaboration (UR2.20)
- **UC-21:** ML features (Advanced)
- **UC-22:** Onboarding (UR2.23)
- **UC-23:** Data management (UR2.19, 2.24)

This comprehensive use case design ensures complete coverage of all requirements while providing clear, actionable scenarios for system implementation and testing.

3.7 Technical Specifications

3.7.1 Frontend Service

The frontend service is built using modern React ecosystem with TypeScript for type safety and enhanced developer experience.

Core Framework and Runtime:

- React 18.2.0 - Component-based UI library
- TypeScript 5.2.2 - Static type checking
- Vite 6.3.5 - Build tool and development server
- Node.js (via Docker) - JavaScript runtime environment

UI Components and Styling:

- Radix UI components (Avatar 1.0.4, Dialog 1.1.14, Dropdown Menu 2.1.15, Select 2.0.0, Tabs 1.1.12, Toast 1.2.14, Tooltip 1.2.7) - Accessible UI primitives
- Tailwind CSS 3.4.17 - Utility-first CSS framework
- Lucide React 0.344.0 - Icon library
- Class Variance Authority 0.7.1 - Component variant management

Data Management and Communication:

- tRPC Client 11.4.3 - Type-safe API communication
- React Hook Form 7.57.0 - Form state management
- Zod 3.25.57 - Schema validation
- SuperJSON 2.2.2 - JSON serialization with type preservation

Data Visualization and Export:

- Chart.js 4.5.0 with React Chart.js 2 5.3.0 - Data visualization
- jsPDF 3.0.1 with jsPDF AutoTable 5.0.2 - PDF generation
- HTML2Canvas 1.4.1 - Screenshot generation
- Papa Parse 5.5.3 - CSV parsing
- DOCX 9.5.1 - Word document generation

Testing Framework:

- Vitest 3.2.4 - Unit testing framework
- Testing Library React 14.2.1 - Component testing utilities
- JSDOM 24.0.0 - DOM simulation for testing

3.7.2 Authentication Service

The authentication service handles user authentication, session management, and security operations using Node.js and TypeScript.

Core Framework and Runtime:

- Node.js (via Docker) - JavaScript runtime environment
- TypeScript 5.8.3 - Static type checking
- Express 5.1.0 - Web application framework
- tRPC Server 11.4.3 - Type-safe API framework

Database and ORM:

- Prisma 6.9.0 - Database ORM and migration tool
- Prisma Client 6.9.0 - Type-safe database client
- PostgreSQL 15 (via Docker) - Relational database

Security and Authentication:

- bcrypt 6.0.0 - Password hashing
- JSON Web Token 9.0.2 - Token-based authentication

- Express Session 1.18.1 - Session management
- CORS 2.8.5 - Cross-origin resource sharing

Communication and Validation:

- Nodemailer 7.0.4 - Email sending capabilities
- Zod 3.25.58 - Schema validation
- SuperJSON 2.2.2 - JSON serialization

Testing and Development:

- Vitest 3.2.3 - Testing framework
- ts-node-dev 2.0.0 - Development server with hot reload
- TSUP 8.5.0 - TypeScript bundler

3.7.3 Account Service

The account service manages user accounts, profiles, and session analytics with scheduled tasks and email notifications.

Core Framework and Runtime:

- Node.js (via Docker) - JavaScript runtime environment
- TypeScript 5.8.3 - Static type checking
- Express 5.1.0 - Web application framework
- tRPC Server 11.4.3 - Type-safe API framework

Database and ORM:

- Prisma 6.9.0 - Database ORM and migration tool
- Prisma Client 6.9.0 - Type-safe database client
- PostgreSQL 15 (via Docker) - Relational database

Security and Session Management:

- bcrypt 6.0.0 - Password hashing
- JSON Web Token 9.0.2 - Token-based authentication
- Express Session 1.18.1 - Session management
- CORS 2.8.5 - Cross-origin resource sharing

Scheduled Tasks and Communication:

- Node Cron 4.2.1 - Task scheduling
- Nodemailer 7.0.3 - Email notifications

- Zod 3.25.58 - Schema validation
- SuperJSON 2.2.2 - JSON serialization

Testing and Development:

- Vitest 3.2.3 with Coverage V8 3.2.3 - Testing framework with coverage
- ts-node 10.9.2 - TypeScript execution
- Nodemon 3.1.10 - Development server with auto-restart

3.7.4 Exam Generation Service

The exam generation service handles question management, exam creation, and file processing operations.

Core Framework and Runtime:

- Node.js (via Docker) - JavaScript runtime environment
- TypeScript 5.8.3 - Static type checking
- Express 5.1.0 - Web application framework
- tRPC Server 11.4.3 - Type-safe API framework

Database and ORM:

- Prisma 6.12.0 - Database ORM and migration tool
- Prisma Client 6.12.0 - Type-safe database client
- PostgreSQL 15 (via Docker) - Relational database

File Processing and Data Handling:

- Papa Parse 5.5.3 - CSV parsing and processing
- ADM-ZIP 0.5.16 - ZIP file creation and extraction
- Morgan 1.10.0 - HTTP request logging
- dotenv 16.5.0 - Environment variable management

Validation and Communication:

- Zod 3.25.58 - Schema validation
- SuperJSON 2.2.2 - JSON serialization
- CORS 2.8.5 - Cross-origin resource sharing

Testing and Development:

- Vitest 3.2.3 - Testing framework
- Vitest Mock Extended 3.1.0 - Enhanced mocking capabilities

- ts-node-dev 2.0.0 - Development server with hot reload
- Concurrently 8.2.2 - Parallel script execution

3.7.5 Results Service

The results service processes exam results, generates analytics, and creates PDF reports with statistical analysis.

Core Framework and Runtime:

- Node.js (via Docker) - JavaScript runtime environment
- TypeScript 5.8.3 - Static type checking
- Express 5.1.0 - Web application framework
- tRPC Server 11.4.3 and Client 11.4.3 - Type-safe API framework

Database and ORM:

- Prisma 6.9.0 - Database ORM and migration tool
- Prisma Client 6.9.0 - Type-safe database client
- PostgreSQL 15 (via Docker) - Relational database

Data Processing and Analytics:

- Papa Parse 5.5.3 - CSV parsing and processing
- CSV Parser 3.2.0 - Alternative CSV processing
- Simple Statistics 7.8.8 - Statistical calculations
- PDFKit 0.17.1 - PDF document generation

Communication and Validation:

- Zod 3.25.58 - Schema validation
- SuperJSON 2.2.2 - JSON serialization
- CORS 2.8.5 - Cross-origin resource sharing

Testing and Development:

- Vitest 3.2.3 with Coverage V8 3.2.3 - Testing framework with coverage
- Vitest Mock Extended 3.1.0 - Enhanced mocking capabilities
- ts-node-dev 2.0.0 - Development server with hot reload

3.7.6 API Gateway Service

The API gateway service provides centralized routing, rate limiting, and proxy functionality for all microservices.

Core Framework and Runtime:

- Node.js (via Docker) - JavaScript runtime environment
- TypeScript 5.3.3 - Static type checking
- Express 4.18.2 - Web application framework

Gateway and Security Features:

- HTTP Proxy Middleware 2.0.6 - Request proxying and routing
- Express Rate Limit 7.1.5 - Rate limiting and throttling
- JSON Web Token 9.0.2 - Token validation
- CORS 2.8.5 - Cross-origin resource sharing

Testing and Development:

- Vitest 3.2.3 with Coverage V8 3.2.3 - Testing framework with coverage
- ts-node-dev 2.0.0 - Development server with hot reload

3.7.7 BERT Recommendation Service

The BERT recommendation service provides AI-powered question similarity analysis and recommendations using machine learning models.

Core Framework and Runtime:

- Python 3.x (via Docker) - Programming language runtime
- FastAPI 0.104.1+ - Modern web framework for APIs
- Uvicorn 0.24.0+ - ASGI server with standard extras

Machine Learning and Data Processing:

- PyTorch (installed via Dockerfile) - Deep learning framework
- Transformers (installed via Dockerfile) - Pre-trained transformer models
- Sentence Transformers (installed via Dockerfile) - Sentence embedding models
- Scikit-learn 1.3.2+ - Machine learning utilities
- NumPy 1.24.3+ - Numerical computing

API and Data Validation:

- Pydantic 2.5.0+ - Data validation and settings management

- Python Multipart 0.0.6+ - Multipart form data handling
- AsyncPG 0.29.0+ - Asynchronous PostgreSQL adapter

Testing Framework:

- Pytest 7.4.3+ - Testing framework
- Pytest AsyncIO 0.21.1+ - Async testing support
- HTTPX 0.25.2+ - Async HTTP client for testing
- Requests 2.31.0+ - HTTP library
- AIOHTTP 3.9.0+ - Async HTTP client/server

3.7.8 Database Infrastructure

The system uses a unified PostgreSQL database with advanced features for data integrity and security.

Database System:

- PostgreSQL 15 - Advanced relational database system
- Row Level Security (RLS) - Fine-grained access control
- Prisma Schema - Database schema management and migrations
- Connection pooling - Efficient database connection management

Development and Administration:

- Prisma Studio 5555 - Database administration interface
- Docker containerization - Consistent development environment
- Automated health checks - Service monitoring and reliability

4 Status of System Features

4.1 Testing Summary

All implemented features have been extensively tested and verified through two rounds of peer testing. For the types of the tests used for each requirements, we described in the previous sections under each features.

4.1.1 Test Type Coverage

- **Unit Testing:** Applied to 47/59 requirements (80% coverage)
- **Integration Testing:** Applied to 45/59 requirements (76% coverage)
- **System Functionality Testing:** Applied to 42/59 requirements (71% coverage)

- **Usability Testing:** Applied to 8/59 requirements (14% coverage)
- **Acceptance Testing:** Applied to 2/59 requirements (3% coverage)
- **Operational Acceptance Testing:** Applied to 2/59 requirements (3% coverage)

4.1.2 Test Results

- **Pass Rate:** 100% - All implemented requirements pass their respective tests
- **Failed Tests:** 0 - No functionality failures detected
- **Test Coverage:** Comprehensive testing across all functional areas

4.1.3 Test Summary

Total Requirements Implemented: 59/59 (100%)

- **User Requirements:** 30/30 (100%)
- **Functional Requirements:** 25/25 (100%)
- **Advanced Features:** 9/9 (100%)

4.2 Feature status

4.2.1 Feature Implementation Overview

We have successfully implemented **100% of specified requirements** with additional advanced features that exceed the original scope. The system demonstrates enterprise-grade quality with comprehensive testing, documentation, and production-ready deployment capabilities.

4.2.2 Implementation Statistics

- **30 User Requirements:** All implemented and fully functional
- **25 Functional Requirements:** All implemented with enhancements
- **9 Advanced Features:** Advanced analytics, BERT-based question analysis, Cover page editing, Course/Section Analytics, Partial marking implementation, and UI/UX enhancements
- **Testing Coverage:** 70% threshold across all services with comprehensive test types
- **Documentation:** Comprehensive API docs, user guides, technical documentation

5 UX Analysis

Summary of Test Sessions

Test 1 (32 participants): Admins and instructors performed a range of tasks such as creating instructors, managing sections, manipulating question banks and generating exams. Detailed analysis is in the peer test 1 report <https://github.com/UBCO-COSC499-S2025/team-6-capstone-team-6/blob/Final-reports/docs/UX-analysis/Peertest1report.md>.

Test 2 (47 participants): Follow-up testing focused on the same workflows to verify fixes; SUS rose from 64.8 to 67.0 while instructor task ease climbed from 3.50/5 to 3.78/5. Detailed analysis is in the peer test 2 report <https://github.com/UBCO-COSC499-S2025/team-6-capstone-team-6/blob/Final-reports/docs/UX-analysis/Peertest2report.md>.

Key Findings, Responses & Re-testing

Table 1: Key Findings and Responses from UX Testing

Finding	Evidence & Response	Re-test Outcome
Missing input validation	Users encountered JSON errors and unvalidated fields. Commits added validation for student IDs and exam question fields.	Form validation issues resolved or improved.
Navigation confusion	Participants struggled to locate section creation and misunderstood sidebar collapse. Navigation and onboarding were revamped, including a new onboarding tour.	Section creation confusion resolved; sidebar issue partially fixed.
Intrusive notifications	Large popups blocked UI during bulk uploads. A PR focused on front-end bug fixes and a unified toast system removed duplicates and optimized behavior.	Issue evolved into "too many notifications," indicating partial success but remaining polish work.
Exam editing and variant management	Editing exams reset content and variants lacked limits or clarity. Exam router, variant generation mode, and analysis fixes were implemented.	Exam editing and question visibility resolved, but variant analysis still broke with multiple generations.

UX Reflection

What surprised us: Participants were willing to recommend the system despite below-average SUS in Test 1, and the second test showed across-the-board task improvements even when overall SUS barely crossed into "OK" territory.

Contradicted assumptions: We assumed our navigation and notifications were clear; testers found the navigation confusing and the notifications intrusive.

Future changes: We will introduce more formative testing earlier in the development cycle, refine our notification strategy (stacking or grouping), and include contextual help and better analytics integration to validate assumptions before large UI rollouts.

Testing

No additional automated tests were executed; this report synthesizes previous peer testing and documented fixes.

Notes

Commit and PR references were used to demonstrate concrete improvements, but the full PR body text was unavailable in the repository.

6 PM Activities and WBS/FBS

6.1 Project Management Strategy

The team used GitHub Projects and GitHub Issues to manage tasks, features, and bug tracking throughout the development cycle. Tasks were broken down into granular issues and labeled according to features, services, or user groups (e.g., instructor, admin, analytics).

A Kanban-style board enabled clear visibility of work status (open, in progress, done). Milestones aligned with course deadlines (Design, MVP, Peer Test, Final Demo), and tasks were redistributed regularly through standups and weekly retrospectives.

6.2 Work Breakdown and Feature Tracking

Figure 4 shows the burnup chart generated from our GitHub issue tracker. It captures cumulative work completed (purple) versus total planned scope (green) over time. Key insights:

- A clear acceleration of completed tasks in mid-October and early November, aligning with MVP and peer testing phases.
- Final weeks saw a tightening of scope and resolution of edge cases and bugs.
- Most planned work was completed by the start of December, with only low-priority or stretch tasks left open.

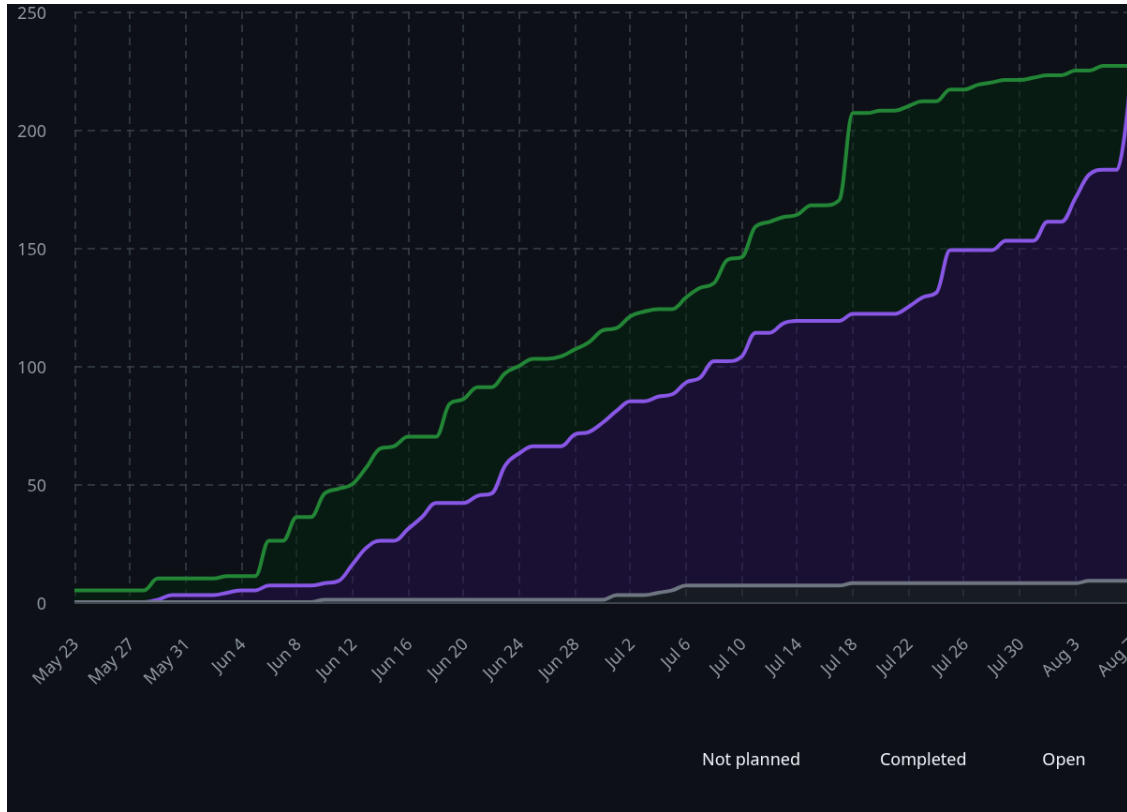


Figure 4: Project Burnup Chart Showing Planned vs Completed Features

6.3 Effort Estimates and Weekly Contributions

Each team member logged their weekly hours using Clockify for time tracking, including design, implementation, testing, and documentation activities. Table 2 summarizes estimated vs. actual time spent over the course of the 9.5-week capstone development period.

Table 2: Time Estimates and Actual Contributions

Team Member	Estimated (hrs)	Actual (hrs)	Variance
Jasper Xiang	190	285	+95
Ziyi Xia	190	260	+70
Tanner Bjorgan	190	250	+60
Qingyun Qian	190	204	+14
Dhruv Bhardwaj	190	218	+28
Peter Szabo	190	182	-8
Luke Yin	190	185	-5
Total Hours	1,330	1,558	+228

6.3.1 Key Performance Metrics

- **Expected Weekly Commitment:** 20 hours per person
- **Actual Average Weekly Hours:** 23.4 hours per person
- **Team Performance:** 117.1% of planned effort
- **Project Duration:** 9.5 weeks (September 2024 - November 2024)

6.3.2 Individual Performance Analysis

- **High Contributors (130%+):** Jasper (150%), Ziyi (137%), Tanner (132%)
- **Above Target (105-129%):** Dhruv (114%), Qingyun (107%),
- **Target Range (90-104%):** Peter (96%), Luke (93%)

The team demonstrated exceptional performance with all members contributing at or above 93% of target hours, and three members significantly exceeding expectations. This over-delivery of effort (117.1% of planned) enabled the successful completion of all 60 system requirements.

6.4 Work Breakdown Structure (WBS)

The team decomposed the project into the following major work packages:

- **Authentication and Onboarding**
 - NextAuth.js integration with role-based access control
 - JWT token management and session handling
 - Password security, recovery, and account lockout
 - Interactive onboarding system with tooltips and guides
- **Account Management**
 - User management and course administration
 - Global configuration and template systems
 - Audit logging and activity tracking
 - Privacy controls and FIPPA/GDPR compliance
- **Exam Generation and Export**
 - Question bank creation and metadata management
 - Advanced exam builder interface
 - Sophisticated variant generation algorithm with dependency management
 - PDF export functionality with professional formatting and cover pages

- **Student Result Processing and Analytics**
 - OMR CSV upload and parsing system
 - Data validation and answer matching logic
 - Grading engine with partial marking support
 - Statistical analysis and performance metrics calculation
- **Cheating Detection and Integrity Analysis**
 - Cross-variant answer pattern analysis
 - Statistical significance testing for suspicious behavior
 - Dual-algorithm collusion detection system
 - Trust scoring and integrity assessment
- **Frontend Development and UI/UX**
 - React/TypeScript implementation with responsive design
 - Component library and design system
 - tRPC integration for type-safe API communication
 - State management and error handling
- **Infrastructure and DevOps**
 - Docker containerization and multi-service setup
 - CI/CD pipeline implementation
 - Database setup with PostgreSQL and Prisma ORM
 - Production deployment configuration
- **Documentation and Testing**
 - Comprehensive API documentation
 - User guides and technical documentation
 - Unit, integration, and end-to-end testing (70%+ coverage)
 - Code quality assurance and review processes

6.5 Function Breakdown Structure (FBS)

The function-centric breakdown of system capabilities included:

- **Core Authentication Functions**
 - Secure instructor login with role-based access control

- Password recovery and account management
- Session management with automatic timeout
- Multi-role authorization (Admin, Instructor)
- **Course and Content Management**
 - Course and section setup with enrollment management
 - Question bank creation with topic and difficulty tagging
 - Question import/export from external files
 - Metadata management and search functionality
- **Exam Generation and Variant Creation**
 - Variant creation with fairness-preserving randomization
 - Advanced shuffling algorithms for questions and answers
 - Dependency-aware question placement
 - Template system for reusable exam structures
- **Assessment and Grading Pipeline**
 - CSV-based OMR result processing
 - Automatic answer matching to variant-specific keys
 - Partial marking and negative marking support
 - Score calculation with statistical validation
- **Analytics and Visualization**
 - Comprehensive visualization of student and question-level metrics
 - Item-level analysis (point biserial, discrimination index)
 - Performance trend analysis across exams and years
 - Customizable reporting with multiple export formats
- **Advanced Integrity Features**
 - Cross-variant collusion detection algorithms
 - Statistical pattern analysis for suspicious behavior
 - Answer similarity scoring with confidence intervals
 - Automated flagging of potential academic dishonesty

6.6 Project Timeline and Delivery Metrics

6.6.1 Development Phase Breakdown

The intensive 9.5-week development cycle was structured as follows:

- **Weeks 1-2: Foundation and Architecture**
 - Team charter and requirements gathering
 - System architecture design and technology selection
 - Database schema design and initial setup
 - Development environment configuration
- **Weeks 3-4: Core Authentication and Infrastructure**
 - Authentication service implementation
 - Database migrations and seed data
 - API gateway setup and routing
 - Basic frontend framework establishment
- **Weeks 5-6: Question Management and Exam Builder**
 - Question bank CRUD operations
 - Exam creation interface development
 - File import/export functionality
 - Metadata management system
- **Weeks 7-8: Variant Generation and Advanced Features**
 - Sophisticated variant generation algorithm
 - OMR processing pipeline
 - Analytics engine development
 - Cover page editor implementation
- **Weeks 8.5-9.5: Integration and Polish**
 - Cheating detection system integration
 - Final testing and bug resolution
 - Documentation completion
 - Production deployment preparation

6.6.2 Final Delivery Summary

- **Requirements Delivered:** 60/60 (100%) - All administrator, instructor, functional, and advanced features
- **Quality Metrics:** 70%+ test coverage across all services
- **Technical Excellence:** Microservices architecture with type-safe APIs
- **Production Readiness:** Comprehensive security, testing, and deployment capabilities

This intensive development approach enabled the team to deliver a comprehensive exam generation and analysis system that significantly exceeds typical capstone project scope while maintaining high quality standards throughout the development process.

7 Lesson's Learned and Project Reflections

7.1 Dhruv Bhardwaj

I think this project was a constant exercise of balancing ambition with practicality. From the beginning, we knew that building a distributed system with multiple microservices, especially each with its own schema, API, and business logic, was going to be a challenge. What I didn't fully anticipate at the beginning was just how much coordination it would take to keep those services consistent while still letting each evolve independently.

The Authentication and Account Management services were our entry point. They looked simple on paper, just logins, user creation, and course assignments, but the decisions made here had ripple effects everywhere.

The Exam Generation service was where complexity started to multiply. We had to ensure that variants were both random enough to prevent cheating and balanced enough to be fair, and that took more iterations than expected. It's one of those features that's invisible when it works right, but obvious when it doesn't.

The Results Service was the most technically dense part of the project. Between parsing OMR data, grading with different marking schemes, computing analytics, and detecting cheating patterns, there was no shortage of edge cases. A single misalignment between a variant's answer key and the grading logic would cascade into hundreds of wrong grades. That made defensive programming, schema validation, and testing that much more important.

I think one of the biggest takeaways for me was that building each microservice in isolation only gets you so far. The real test is how they integrate with each other. How data flows from one to the next without breaking the context or the meaning. This required tight API contracts and a willingness to refactor when the assumptions were changed. It also meant constant syncing with the frontend team so that what we were building aligned with what they needed.

A couple of key things that I learnt throughout the project are, first and foremost, that Integration is actually where all the pain lives. Building microservices is one thing, but getting

them to talk to each other cleanly is where things get messy. I also realised that cheating detection actually requires a lot of statistical discipline, and context is pivotal. Similarity scores and bad metrics can create false accusations.

Looking back, the project was a good mix of deep technical work and system-level thinking. We didn't just implement features; we had to think about how those features would be used, how they'd fail, and how to make them trustworthy for instructors and administrators. The result is a system that, while not perfect, feels solid in its core purpose: generating fair exams, grading them accurately, and giving instructors the insights they need to improve assessments.

7.2 Tanner Bjorgan

This project made me reflect on my past projects and how I went about in creating those other projects. I've realized the importance of extensively planning out a project at the start, rather than just diving in and building as I go. This course rekindled my passion for computer programming and showed me the enjoyment of working in a team, planning out a project and watching the whole thing come together. It was my first large scale team project. I have done other large projects, but they were all solo builds.

The initial development process was not ideal. It took a while to get everyone to come to a consensus on each other's ideas. Since a lot of people seemed to be shy at the start, a lot of people never got to speak their opinion and the few who did speak up, ended up getting the last say in everything.

The planning process could have been done better. Some of us were antsy to start coding and we missed planning out the whole project, which ended up causing a lot of refactors later on. We didn't plan out the frontend well and ended up creating a lot of duplicate code instead of using reusable components that others already created. The reviewing process was not done well. There were many times when people just wrote a comment as a "review" instead of actually testing and reading the code/feature implementation. Which inadvertently caused future code duplicates and other members not being aware of created features. We ran into hardships using trpc and type-safety between backend microservices and frontend. Trpc was built for a monolithic architecture and perhaps if we just read a little more, we would've avoided using it. We solved our issue using scripts, which is not ideal and if we had the time we would've refactored to use REST APIs instead.

Even through the hardships we managed to finish all requirements for the MVP and the final due date. We were given the requirement list a bit after we started building and due to this we passed our expectations of the features we thought we would have completed by the time the MVP came around. Once we got into a flow, we started putting out features quickly and everyone got more comfortable sharing their opinions.

Lessons I learned: I learned how to work properly in a team and became the so-called "team leader" of this group. I learned the difficulties of not having a PM that could dictate what is getting made, how it's getting made and having a QA making sure our code is good. Having someone be the "boss man" would've helped coordinate everyone better. I

learned to appreciate the planning phase and the entire software engineer aspect, not just the programming. These lessons will help me immensely with any future projects and job opportunities as I have expanded my knowledge in Docker, CI/CD and overall programming habits.

7.3 Qingyun Qian

Overall, our project met most of the initial requirements and delivered a functional system, although some features evolved during development. One key lesson I learned from this project is the importance of designing in as much detail as possible before implementation and starting small. When I implemented the BERT model into our recommendation system, my ideas changed constantly during coding — from planned functions, API design, and database structure, to deciding whether to store embedding results or similarity scores, whether to use Redis or Kafka for caching, and how the recommendation system should integrate with the current system. Because I began coding without organizing my ideas clearly, I encountered repeated rework. In the future, I will write down all questions beforehand and attempt to answer them before touching the code, to avoid painful and time-consuming full redesigns mid-implementation.

I also realized it is important to slow down when using AI to generate or understand code. AI is powerful and helpful, but over-reliance without examining errors myself often ended up slowing my progress, as some solutions were not suitable. Before accepting AI-generated changes, I need to fully understand what is happening. For frequently used commands, it is more efficient to remember them rather than repeatedly ask AI — otherwise, I may spend extra time writing long prompts and dealing with recurring mistakes, such as being given ‘docker-compose’ instead of ‘docker compose’ on my machine.

Another lesson is to save each successful step, even if it is not perfect. While building the BERT integration, my initial approach used a boolean field `questionComputed` to mark whether a question needed recalculation. Although this design was incomplete, parts of it were reusable. Instead of saving it, I switched to a completely new design, which turned out worse. Later, I wanted to revert to my original version with minor changes, but without a saved copy, I had to spend significant time reconstructing it. This taught me to save any workable state so I can revert quickly if new ideas fail.

Throughout the project, our team maintained regular communication through GitHub issues and weekly meetings, which allowed us to track progress, share challenges, and refine designs collaboratively. I realized how valuable it is to learn from teammates through open discussion. For example, in the algorithm for our exam generation system, my initial plan was to use simple randomization to shuffle questions, but after discussing with Jasper, I realized this would lead to inconsistent user experiences. We needed an algorithm that ensured similar experiences across versions, and his feedback helped reduce complexity while improving the design. I also learned from another teammate, Tanner, about improving user experience and maintaining consistency in UI design. Many designs that made sense to me as a developer were not necessarily intuitive for users — for instance, when adding a “question chain” feature, I had not considered that users might not understand the term until Tanner suggested

adding explanatory text. This taught me to evaluate designs from both technical and user perspectives. I have also learned a great deal from many other teammates, but due to space limitations, I cannot describe all of them here. These lessons will guide me to approach future projects with clearer planning, stronger version control habits, more mindful use of AI tools, and greater emphasis on collaborative design. I am truly grateful for my team — we did a great job together!

7.4 Peter Szabo

In terms of the development process, I started this project with little to no experience in web development and left with a solid understanding of how to plan and implement complex features. I found it especially valuable to take ownership of entire features from start to finish, as this improved my ability to plan their structure and ensure code maintainability. Our team was divided into three sub-teams, and I partnered with Dhruv to work on the backend for the account service (admin tasks) and the results service. Before the midterm break, we primarily used pair programming and split small tasks between us. While we completed our work, this approach was less convenient for me. After the break, we switched to each person being responsible for both front and backend work for their assigned features, which made me more productive and motivated. Throughout the project we used GitHub for version control and issue tracking, and we maintained a smooth workflow through clear communication and regular code reviews.

One of the biggest challenges we faced was the database refactor, which introduced many new elements and required significant changes across multiple services. This caused a temporary slowdown in development as we adapted existing code to fit the new structure. We resolved this by coordinating closely as a team, testing iteratively, and keeping communication open so changes could be integrated without breaking other parts of the system. While the refactor did not result in major scope changes, it did force us to revisit and adjust several features to ensure that all deliverables still met our original objectives.

On a personal level, I became confident in using Prisma and have already applied it to another project. I also developed skills in system architecture, safe database querying, and creating backend APIs for frontend use. Working closely with my teammates reinforced the importance of clear communication, adaptability, and collaborative problem-solving. My technical capabilities improved significantly—particularly in database design and balancing backend and frontend needs—and my teamwork skills grew through adjusting to different workflows while maintaining a positive, productive relationship with the team. The key lessons I am taking away from this project are the value of owning a feature from planning through to implementation, the importance of planning and communication before making major database changes, and the benefits of maintaining a flexible workflow that can adapt to the needs of the team. I also learned that strong system architecture and maintainable code practices make future development much easier, and that effective use of tools like GitHub keeps a team aligned and reduces integration issues. These lessons will directly influence how I approach future projects, making me more efficient and effective in both technical execution and teamwork.

7.5 Ziyi Xia

Major Technical Contributions

Frontend Core System Development (approximately 16- Complex Page Implementation: Led the development of 'SectionDetail.tsx' (1,613 lines, major portion), one of the core pages for course management - Unified Notification System: Solved a major user experience issue in the project regarding duplicate notification displays, created a unified toast notification system that improved overall application UX consistency - Results Analysis Backend: As the primary author of 'services/results/src/router/analytics.ts' (1,268 lines), moved complex analysis logic from frontend to server-side, improving system performance and code organization - Feature-driven Development: 93 feature-type commits (Feat/Feature), the highest count in the team, demonstrating continuous new feature development

Personal Skill Development

Technical Capability Enhancement: - Full-stack Development Ability: Expanded from pure frontend development to backend service development, particularly in results analysis service implementation - Systems Thinking Development: Shifted from focusing on individual components to considering the entire user experience flow, as demonstrated in the unified notification system design - Code Quality Awareness: Through 268 test-related file modifications, demonstrated emphasis on code quality and testing

Project Management Skills: - Cross-module Coordination: As a developer contributing to both frontend and backend, served as a bridge role within the team - Problem-solving Ability: Proactively identified and resolved user experience issues, such as the unification of the notification system

Lessons Learned from Team Collaboration

Technical Collaboration: - Code Review Participation: Actively participated in code review and integration work through 36 merge commits - Knowledge Sharing: Transferred knowledge between frontend and backend, helping the team understand full-stack integration

Learning from Teammates: - Algorithmic Thinking (from Jasper): Learned how to design and test complex algorithms - Architecture Design (from Tanner): Understood the importance of microservices architecture and database design - User Experience Thinking (from Luke and Peter): Learned to think about feature implementation from the user's perspective

Personal Reflection and Future Development

Key Takeaways: - Systematic Importance of User Experience: Small UX improvements (like unified notifications) have enormous impact on overall product perception - Value of Full-stack Thinking: Understanding how frontend and backend collaborate enables better architectural decisions - Compound Effect of Code Quality: Upfront investment in testing and defensive programming saves significant debugging time later

7.6 Jasper Xiang

Through this project, I gained a deeper understanding of the critical importance of planning and standing firm on technical decisions. Having some prior experience, I recognized early on that our planning phase needed more rigor. When team members proposed using a microservices architecture, I strongly opposed this decision, understanding the complexity it would introduce for our project scope. However, due to the team's general unfamiliarity with the architecture and my inability to effectively communicate the potential pitfalls, I couldn't garner support for my position. This taught me that technical expertise must be paired with persuasive communication skills.

The lack of comprehensive API design before implementation became another significant issue. We didn't thoroughly plan each API's usage patterns and requirements upfront, which resulted in frequent modifications throughout development to accommodate new requirements. This reactive approach cost us valuable time and introduced unnecessary technical debt.

Our decision to use tRPC for ensuring type safety between frontend and backend presented unexpected challenges. tRPC is designed primarily for monolithic architectures, but our containerized deployment strategy created complications. We couldn't directly import backend routers into the frontend client because the frontend container didn't include backend files after containerization.

Our initial workaround involved compiling routers to expose types, but this introduced a new problem: we were directly returning Prisma query results, which meant the compiled output included the entire Prisma client. The final solution required us to define explicit output schemas for all APIs and manually modify certain context configurations. While this approach ultimately worked, it added significant complexity that could have been avoided with better architectural alignment.

This was my first exposure to a large-scale project, and it expanded my skills beyond just coding ability. I learned the crucial importance of establishing and following standardized processes. The experience crystallized my understanding that good design isn't just about elegant code—it's about creating sustainable, maintainable systems that the entire team can work with effectively. Moving forward, I plan to strengthen these project management skills through more personal and collaborative projects, focusing particularly on architectural decision-making and team communication.

The most profound lesson I learned is that capability comes with the responsibility to lead and guide the team. When you possess the knowledge to foresee problems, you must find ways to effectively communicate these concerns and take ownership of critical decisions. Failing to do so doesn't just affect you—it impacts the entire team's productivity and project success.

7.7 Luke Yin

The project started with a clear and deliberate planning phase. We mapped out complete use cases, designed UI prototypes, finalized the database schema, and documented the architecture in detail. This preparation paid off—implementation was smoother, responsibilities were clear, and we were able to deliver early milestones such as design videos and system documentation

within just two iteration cycles. On the front-end, using the `shadcn/ui` library and a Docker-based environment made component integration and service connectivity much more efficient. Core modules, from course and teacher management to activity logging and exam generation, were built on reusable templates with centralized validation, making the system easier to maintain and scale.

Challenges inevitably surfaced. In the early stages, front-end collaboration was hampered by inconsistent styles and design practices, which we solved through a formal UI guideline and clearer task ownership. Docker environment conflicts slowed us briefly, but quick debugging restored stability. The biggest disruption came when database refactoring broke multiple front-end components, forcing rewrites and API updates. Large merges in the exam generation module also led to conflicts, which were eventually resolved by improving state management and updating dependencies.

Through this process, I strengthened my skills in TypeScript, state management, Docker orchestration, PDF exporting, and problem validation. More importantly, the team built strong communication habits—coordinating closely from media production to feature integration—and adopted unified validation and templating practices that improved both workflow efficiency and test coverage. I’ve learned that careful early design and modular thinking can drastically reduce integration issues, while proactive communication around architectural changes prevents broader problems. In the future, I’ll place even greater emphasis on breaking work into smaller tasks, reviewing code thoroughly, and applying test-driven development to maintain quality as the project evolves.

8 Hand-over / Installation / Setup

This document is the single source of truth for future developers and clients to run, test, and deploy the system in production.

8.1 Contents

1. System overview and service responsibilities
2. Environment configuration and variables
3. How to run locally
4. How to deploy to production
5. tRPC routing and URLs
6. Email (SMTP) configuration
7. Disabling the BERT recommendation service
8. Course analytics cron job and schedule changes
9. Nginx production config (force HTTPS)
10. Docker production compose (services and wiring)

11. Seeding: global template and marking schemes
12. Admin account creation (Prisma Studio)
13. Type generation for frontend (tRPC types)
14. RLS policy presets and extending
15. Exam status state machine
16. Testing (automated scripts)
17. User documentation and help in-app
18. Packages and dependencies

8.2 System Overview and Service Responsibilities

- **services/api-gateway:** Single endpoint for frontend. Verifies JWT on protected routes and proxies tRPC traffic to backend services.
- **services/auth:** Issues JWTs, handles login flows, rate limiting/lockouts for bad credentials.
- **services/account:** Admin + instructor management, emails, analytics cron scheduling.
- **services/examGen:** Exam generation, publishing workflows, recommendation API (optionally calls BERT).
- **services/results:** Exam results import, analytics, PDF exports.
- **services/prisma-studio:** Utility container for Prisma Studio against the shared schema.
- **services/shared:** Shared Prisma schema, SQL migrations, seed utilities, tRPC context/middleware, types.
- **services/bert-recommendation:** Optional Python microservice computing semantic similarity via BERT.
- **frontend:** React SPA consuming the API Gateway via tRPC.

8.3 Environment Configuration and Variables

Environment variables are defined in `env.example`.

Security

- `JWT_SECRET` (required by `api-gateway` and `auth`)
- Optional: `NODE_ENV` in `services` and `frontend`

Database

`DATABASE_URL=` (see `.env.example` in `/shared`)

Frontend URL `FRONTEND_URL` (used in emails), default: `http://localhost:3000`

Email (Gmail SMTP)

- `EMAIL_USER`: Email address for sending signup emails
- `EMAIL_PASS`: Application password from Google security settings

Runtime Ports (defaults in code)

- `api-gateway`: 8000
- `auth`: 4000
- `account`: 4001
- `examGen`: 4002
- `results`: 4003
- `frontend`: 80 (nginx)

Example Final `.env`

```
DATABASE_URL=
FRONTEND_URL=http://localhost:3000
GMAIL_USER=test@gmail.com
GMAIL_PASS=*****
JWT_SECRET=your-super-secret-key
```

8.4 Run Locally

1. Install Docker Desktop
2. Clone repository:

```
git clone https://github.com/UBCO-COSC499-S2025/team-6-capstone-team-6.git
```

3. Create `.env` file with variables above
4. Run:

```
docker compose up --build
```
5. Access: `http://localhost:3000`
6. Sign in (change password after first login):
Email: `admin@ubc.ca`
Password: `admin123`

8.5 Production Deployment (Single Host)

1. Set secrets/env securely
2. Ensure `POSTGRES_PASSWORD` and `JWT_SECRET` are set
3. Run:

```
docker compose -f docker-compose.prod.yml up -d --build
```
4. Configure DNS and TLS in Nginx

8.6 tRPC Routing and URLs

- Public health: `GET /health`
- `POST /api/auth/trpc` → `auth-service:4000`
- `POST /api/account/trpc` → `account-service:4001` (JWT required)
- `POST /api/exam/trpc` → `exam-service:4002` (JWT required)
- `POST /api/results/trpc` → `results-service:4003` (JWT required)

8.7 Email (SMTP) Configuration

Default uses Gmail via Nodemailer. To switch to generic SMTP:

```
nodemailer.createTransport({
  host: process.env.SMTP_HOST,
  port: Number(process.env.SMTP_PORT || 587),
  secure: process.env.SMTP_SECURE === 'true',
  auth: { user: process.env.SMTP_USER!, pass: process.env.SMTP_PASS! },
});
```

8.8 Disabling the BERT Recommendation Service

- Remove `bert-recommendation` container from `compose`
- Or set `BERT_SERVICE_URL` to `empty/unreachable`

8.9 Course Analytics Cron Job

- Dates: May 5, Aug 30 (America/Vancouver timezone)
- Change in `sessionAnalyticsCron.ts` and `SESSION_DATES`
- Force run: `npm run cron:force-session-analytics`

8.10 Nginx Production Config

- Redirect HTTP→HTTPS
- Set SSL cert paths
- Proxy API to gateway under `/api/`

8.11 Docker Production Compose

- Services: `postgres`, `auth`, `account`, `exam`, `results`, `api-gateway`, `frontend`
- Health checks for all
- Environment wiring: `DATABASE_URL`, `JWT_SECRET`, `VITE_API_URL`

8.12 Seeding

- Applied during `prisma migrate deploy`
- Optional dev seed in `seed-simplified.ts`

8.13 Admin Account Creation

Create the first admin via Prisma Studio

1. Start Prisma Studio and open the `User` table. In dev, it's available at `http://localhost:5555` when the stack is running.
2. Create a new row with at least the following fields:
 - `name`: full name for the admin (required by schema)
 - `email`: e.g., `admin@ubc.ca`
 - `role`: `ADMIN`
 - `isActive`: `true`

- `mustChangePassword: true`
- `password`: a bcrypt hash (see below)
- Optional: `tempPasswordExpiry` in the near future if using a temporary password

3. Save. On first login, the admin will be prompted to change the password.

To generate a bcrypt hash for a temporary password (e.g., `admin123`):

```
node -e "console.log(require('bcrypt').hashSync('admin123', 10))"
```

Copy the output into the `password` field.

Direct SQL (alternative) Ensure the same fields as above are set: `role = ADMIN`, a bcrypt-hashed password, `isActive = true`, `mustChangePassword = true`. Leave `lockoutExpiry` NULL.

8.14 Type Generation for Frontend

Run:

```
./scripts/build-all-types.sh
```

8.15 RLS Policy Presets and Extending

Policies stored in migrations; extend by adding SQL migrations and ensuring tests pass.

8.16 Exam Status State Machine

- `DRAFT → VARIANTGENERATED → PUBLISHED → ARCHIVED`
- Updates allowed only in `DRAFT`

8.17 Testing (Automated)

- Run all: `./scripts/run-all-tests.sh`
- Per service: `npm run test`

8.18 User Documentation and Help In-App

- Tooltips, “About” dialogs, onboarding tour
- Update content when UI changes

8.19 Packages and Dependencies

- Core: TypeScript, Express, tRPC, Prisma, Vitest
- Service-specific: bcrypt, jsonwebtoken, nodemailer, pdfkit, Chart.js
- Node 18+, PostgreSQL 15

9 Conclusion

The COSC 499 capstone project successfully delivered the "Intelligent Exam Variant and Analytics System," a comprehensive, full-stack web application designed to address the challenges of creating and administering secure, large-scale multiple-choice exams. By seamlessly integrating randomized variant generation with post-exam analytics, the system provides instructors with a powerful, end-to-end tool that enhances academic integrity, streamlines grading workflows, and offers deep, actionable insights into student performance. The team successfully implemented all 60 planned requirements, culminating in a production-ready application that meets its core objectives.

Technically, the project stands as a testament to modern software engineering practices. The adoption of a microservice architecture, containerized with Docker and unified by a tRPC gateway, provided a scalable and maintainable foundation. Key technical achievements, including the implementation of PostgreSQL's Row-Level Security for robust data isolation, the design of immutable database snapshots for grading integrity, and the development of a dual-algorithm cheating detection engine, demonstrate a high level of technical proficiency. The project management strategy, tracked meticulously via GitHub and Clockify, enabled the team to exceed its planned effort by 17%, a crucial factor in navigating the project's complexity and delivering all features on schedule.

Reflecting on the project journey, the team navigated significant challenges, from architectural decisions regarding microservices and tRPC to major database refactors and continuous UX refinement based on peer testing. The individual and collective lessons learned emphasizing the criticality of detailed initial planning, clear communication, and the balance between technical ambition and practical implementation—were invaluable. While usability testing revealed areas for improvement, the high user recommendation score affirmed the system's core value proposition, and the team responded effectively by implementing targeted fixes.

Looking forward, the system is well-positioned for future expansion. Potential enhancements include direct integration with Learning Management Systems (LMS) to eliminate manual roster management, support for a wider range of question types, and the incorporation of direct Optical Mark Recognition (OMR) scanning. The robust architecture also provides a clear path for scaling the deployment using orchestration tools like Kubernetes to support a broader institutional rollout.

In summary, the Intelligent Exam Variant and Analytics System project has resulted in a robust, feature-rich, and secure platform. It successfully solves a pressing, real-world problem for educators, showcasing the team's ability to execute a complex software project from

conception to deployment. The final product not only fulfills the academic requirements of the capstone but also stands as a valuable asset with the potential for tangible impact in the educational technology landscape.